

PawsPlan – APLICAȚIE DE MONITORIZARE A TRATAMENTELOR ANIMALELOR DE COMPANIE

Candidat: Bianca-Maria BELEA

Coordonator științific: Șl. Dr. Ing. Cosmin MARȘAVINA

Sesiunea: Iunie 2023

REZUMAT

„PawsPlan” este un proiect ce își propune să vină în ajutorul posesorilor de animale de companie. Acesta permite înregistrarea medicației prescrise de medicul veterinar (tratamente, vaccinuri, deparazitări), dar și notificarea utilizatorilor prin e-mail atunci când medicația are nevoie de înnoire.

Odată înregistrat, un utilizator își poate adăuga animalele în aplicație, introducând date referitoare la numele, vârsta și rasa animalului, poate edita și șterge aceste informații.

Ulterior, utilizatorul are acces la o listă simplificată a animalelor, ce conține doar numele acestora, pe care, dacă îl selectează, are la dispoziție lista medicațiilor animalului respectiv, dar și un card cu toate informațiile despre necuvântător.

În cadrul listei de medicații, există posibilitatea de a adăuga o nouă intrare pentru animalul respectiv, dar și de a o elimina.

În urma notificării primite prin email, utilizatorului i se pun la dispoziție date de contact ale veterinarilor înregistrați în aplicație.

Aplicația este destinată utilizării web. Pentru dezvoltarea componentei de front-end am folosit ReactJS, o bibliotecă a limbajului JavaScript, iar pentru componentele de server de back-end și baza de date am folosit serviciile platformei Firebase.

Pentru a trimite utilizatorilor notificările prin email, aplicația folosește o librărie a limbajului JavaScript, numită EmailJS, care, pentru comunicarea dintre servere, folosește protocolul simplu de transfer al corespondenței (SMTP).

ABSTRACT

“PawsPlan” is a project that aims to help pet owners. It allows the registration of prescribed medication by a veterinarian (treatments, vaccines, parasite preventions), as well as notifying users by e-mail when the medication needs renewal.

Once the user is registered, they can add their pets to the application by providing information about the animal, such as name, age and breed or they can edit or delete all this information.

After that, the user has access to a simplified list of their animals, which has only their names. When selecting a name, the user can view the medication list for that particular animal, as well as a card with all the information about the pet.

Within the list of medication, there is an option to add a new entry for that specific pet or remove an existing one.

After receiving the email notification, the user is provided with contact information of registered veterinarians within the application.

The application is designed for web usage. For the development of the front-end component, I utilized ReactJS, which is a JavaScript library, while the components for the back-end server and database were built with services provided by Firebase platform.

To send email notifications to users, the application uses another JavaScript library called EmailJS, which utilizes the Simple Mail Transfer Protocol (SMTP) for communication between the two servers.

CUPRINS

1. INTRODUCERE	6
1.1. CONTEXT	6
1.2. MOTIVAȚIE	6
1.3. STRUCTURA LUCRĂRII	6
2. STADIUL ACTUAL ÎN DOMENIU	8
2.1. APLICAȚII ASEMĂNĂTOARE	8
2.2. IMPACTUL NOTIFICĂRIILOR ASUPRA UTILIZATORILOR	10
3. TEHNOLOGII UTILIZATE	12
3.1. HTML	12
3.2. CSS	13
3.3. JAVASCRIPT	14
3.4. REACTJS	15
3.5. FIREBASE	18
3.6. WEBSTORM	21
3.7. VERSIONAREA CODULUI. GIT	21
3.8. BIBLIOTECI ALE REACTJS	21
4. PROIECTAREA APLICAȚIEI	23
4.1. FUNCȚIONALITĂȚILE APLICAȚIEI	23
4.2. DIAGRAMA UML A CAZURILOR DE UTILIZARE	25
4.3. DIAGRAMA DE FLUX AL UNUI UTILIZATOR	26
4.4. PROIECTAREA BAZEI DE DATE	27
5. IMPLEMENTARE	29
5.1. ÎNREGISTRAREA UTILIZATORULUI	29
5.2. AUTENTIFICAREA UTILIZATORULUI	30
5.3. VIZUALIZAREA LISTEI DE ANIMALE	30
5.4. ADĂUGAREA UNUI ANIMAL	32
5.5. EDITAREA UNUI ANIMAL	34
5.6. ȘTERGEREA UNUI ANIMAL	34
5.7. VIZUALIZAREA MEDICAȚIILOR PENTRU UN ANIMAL	34
5.8. ADĂUGAREA UNEI MEDICAȚII PENTRU UN ANIMAL	36
5.9. ȘTERGEREA UNEI MEDICAȚII PENTRU UN ANIMAL	37
5.10. LISTA VETERINARILOR PARTENERI	37
5.11. NOTIFICAREA PRIN EMAIL A UTILIZATORULUI	38
5.12. GĂZDUIREA ONLINE A APLICAȚIEI	39
6. UTILIZAREA APLICAȚIEI	41
6.1. ÎNREGISTRAREA UTILIZATORULUI	41
6.2. AUTENTIFICAREA UTILIZATORULUI	42
6.3. VIZUALIZAREA LISTEI DE ANIMALE	42
6.4. ADĂUGAREA UNUI ANIMAL	43
6.5. EDITAREA UNUI ANIMAL	43

6.6.	ȘTERGEREA UNUI ANIMAL	44
6.7.	VIZUALIZAREA MEDICAȚIILOR PENTRU UN ANIMAL.....	45
6.8.	ADĂUGAREA UNEI MEDICAȚII PENTRU UN ANIMAL	45
6.9.	ȘTERGEREA UNEI MEDICAȚII PENTRU UN ANIMAL	46
6.10.	LISTA VETERINARILOR PARTENERI	46
6.11.	NOTIFICAREA PRIN EMAIL A UTILIZATORULUI.....	46
7.	CONCLUZII.....	48
7.1.	CONCLUZII	48
7.2.	DIRECȚII DE DEZVOLTARE	48
	BIBLIOGRAFIE	50
	LISTA FIGURILOR ȘI A TABELELOR.....	52

1. INTRODUCERE

1.1. CONTEXT

Animalele de companie sunt o prezență importantă în viețile noastre, venind cu numeroase beneficii, de la micile bucurii cu care acestea ne încântă zilnic, la responsabilizare sau chiar îmbunătățirea sănătății noastre fizice și mentale.

Un studiu desfășurat în campusul Universității Pepperdine arată că animalele de companie, mai specific câinii de terapie au avut un rol important în scăderea nivelului de stres al studenților în perioada examenelor, astfel cei care au luat parte la studiu au obținut note mai mari la examene decât cei neimplicați [5].

Odată cu avantajele pe care le oferă posesia unui animal de companie, trebuie să înțelegem că acesta vine cu o responsabilitate mare, având nevoie de o îngrijire corespunzătoare. Monitorizarea sănătății animalului poate fi diferită de cea a oamenilor, fiind nevoie de o comunicare constantă cu medicul veterinar, mai ales în primul an de viață.

1.2. MOTIVAȚIE

Trăind în era vitezei, oamenii au numeroase activități de dus la bun sfârșit într-o zi, unele din ele fiind chiar suprapuse ca momente de timp. Cu un program așa încărcat, dacă un posesor de animal de companie nu este o persoană foarte organizată, poate ușor să treacă cu vederea anumite aspecte ce țin de viața necuvântătorului, precum un control la veterinar.

Motivația realizării acestei aplicații vine ca o nevoie reală pe care am avut-o. Ca posesoare a trei animale de companie, este dificil să rețin toate datele vaccinurilor sau deparazitărilor, care sunt notate în trei carnete de sănătate diferite. Astfel, am simțit nevoia de a avea toate aceste informații centralizate într-un singur loc, digital, pentru a nu ajunge în situația în care unul din animalele mele să nu fie la zi cu medicațiile necesare.

„PawsPlan” vine în ajutorul persoanelor care vor să fie mai organizate în gestionarea animalelor de companie, toate acestea ducând la menținerea sănătății acestora.

1.3. STRUCTURA LUCRĂRII

În continuare, se va prezenta structura acestui document. Pentru început, în capitolul „Stadiul actual în domeniu”, am analizat și comparat aplicații asemănătoare cu aplicația „PawsPlan” din punct de vedere al domeniului și al funcționalităților. În plus, am prezentat impactul notificărilor asupra utilizatorilor în funcție de tipul acestora.

În capitolul 3 sunt prezentate tehnologiile utilizate și limbajele de programare folosite pentru a crea aplicația, dar și justificarea folosirii acestora.

Capitolul „Proiectarea aplicației” conține informații referitoare la pașii premergători dezvoltării aplicației. Înainte de a începe scrierea efectivă a codului, a fost nevoie de formularea unor cerințe din perspectiva utilizatorului, dar și de explicarea utilizării aplicației, cu ajutorul diagramelor de cazuri de utilizare și de flux. Am continuat apoi cu prezentarea structurii datelor în baza de date.

Capitolul 5 se concentrează pe implementarea efectivă a codului, incluzând fragmente importante din codul sursă al aplicației, din fiecare funcționalitate a acesteia, la care se adaugă explicații suplimentare.

Perspectiva utilizatorului este prezentată în capitolul „Utilizarea aplicației” prin descrierea scenariilor de utilizare, cu ajutorul ecranelor din aplicația web.

În ultimul capitol am inclus concluziile referitoare la dezvoltarea aplicației, contribuțiile proprii aduse, dar și provocările întâmpinate în întregul proces. Am încheiat cu prezentarea unor direcții viitoare de dezvoltare, care pot îmbunătăți aplicația „PawsPlan”.

2. STADIUL ACTUAL ÎN DOMENIU

2.1. APLICAȚII ASEMĂNĂTOARE

Pentru a compara aplicația dezvoltată, am căutat alte aplicații, atât web, cât și pentru telefonul mobil pentru gestionarea sănătății animalelor de companie.

„VitusVet” este o aplicație mobilă ce vine în ajutorul posesorilor de animale în mai multe direcții, precum acces la rezultatele analizelor, programări sau urmărirea medicației sau a greutății și a activității animalului.



Figura 1 - Profilul animalului de companie în aplicația „VitusVet”

„Furiend – Pet Health Tracker” este destinată telefoanelor mobile și oferă următoarele funcționalități: urmărirea meselor, a greutății și temperaturii animalului, programări și urmărirea medicației.

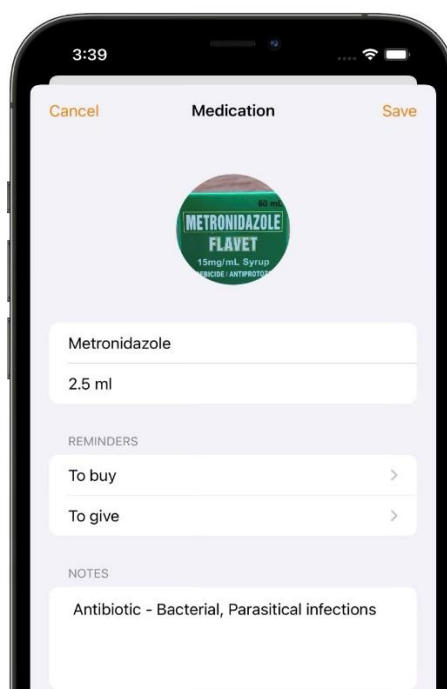


Figura 2 - Medicația pentru un animal de companie în aplicația „Furiend – Pet Health Tracker”

Pe lângă toate funcționalitățile pe care un utilizator le poate accesa în aplicațiile „VitusVet” și „Furiend – Pet Health Tracker”, aplicația „PawsPlan” aduce o îmbunătățire în ceea ce privește notificarea utilizatorului atunci când medicațiile necesită înnoire, deoarece notificările sunt transmise prin e-mail și nu se pot pierde la fel de ușor prin numărul mare de notificări de tip push care vin zilnic pe un telefon mobil.

Tabel 1 - Analiza aplicațiilor „VitusVet” și „Furiend – Pet Health Tracker” comparativ cu „PawsPlan”; funcționalitățile prezente sunt marcate cu ✓, iar cele absente cu ✗

Caracteristici	VitusVet	Furiend – Pet Health Tracker	PawsPlan
Autentificare	✓	✓	✓
Înregistrare a mai multe animale	✓	doar în varianta contra cost	✓
Înregistrare medicații	✓	✓	✓
Detalii despre medici veterinari	✓	✗	✓
Programări	✓	✓	✗
Notificări	push	push	e-mail

2.2. IMPACTUL NOTIFICĂRILOR ASUPRA UTILIZATORILOR

Cele mai folosite tipuri de notificări în crearea de aplicații sunt cele de tip „push” și cele prin e-mail. Acestea reprezintă unele din cele mai folosite ustensile de marketing, fiecare având propriile avantaje și dezavantaje. Nu există o anume „cea mai bună metodă” de notificare a utilizatorilor, ci există o „ce mai potrivită metodă” pentru o anumită platformă [13].

Notificările „push” sunt mesaje clicabile ce pot apărea în interfața cu utilizatorul, indiferent de natura acestora. Cuprind, în general, mesaje scurte și cu un impact puternic, ce fac utilizatorul să acceseze notificarea, fie în browser, fie pe desktop, pe dispozitivele mobile sau chiar pe accesorii precum ceasurile inteligente [14].

Aceste notificări sunt alcătuite din mai multe elemente esențiale pentru atingerea scopului lor, acela de a atrage utilizatorul. Mai jos este prezentă o schemă a „anatomiei” unei notificări push, dar putem aminti titlul, mesajul atractiv, imaginea și URL-ul [14].

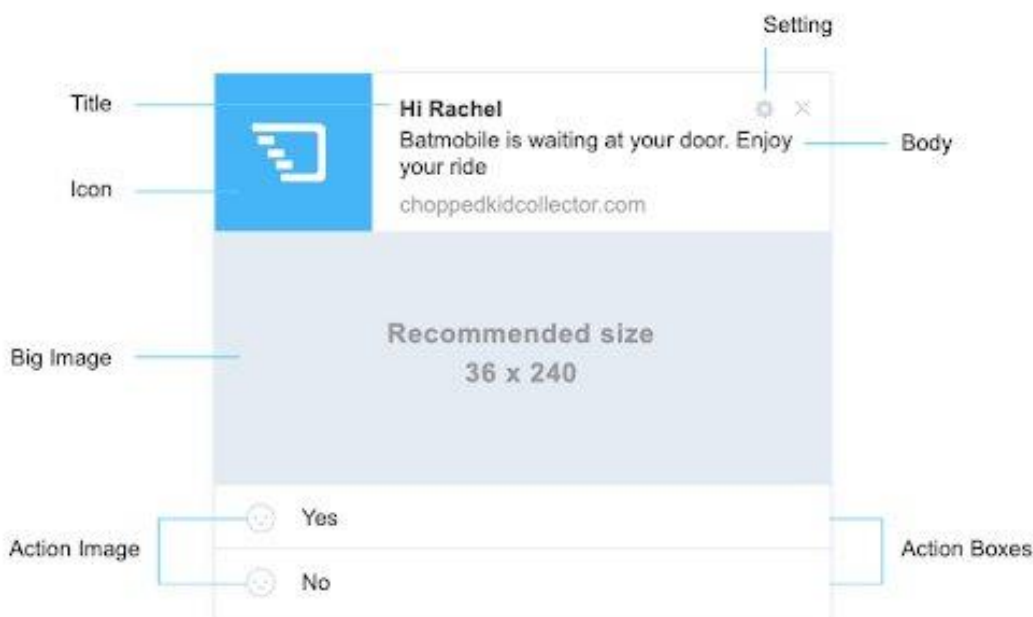
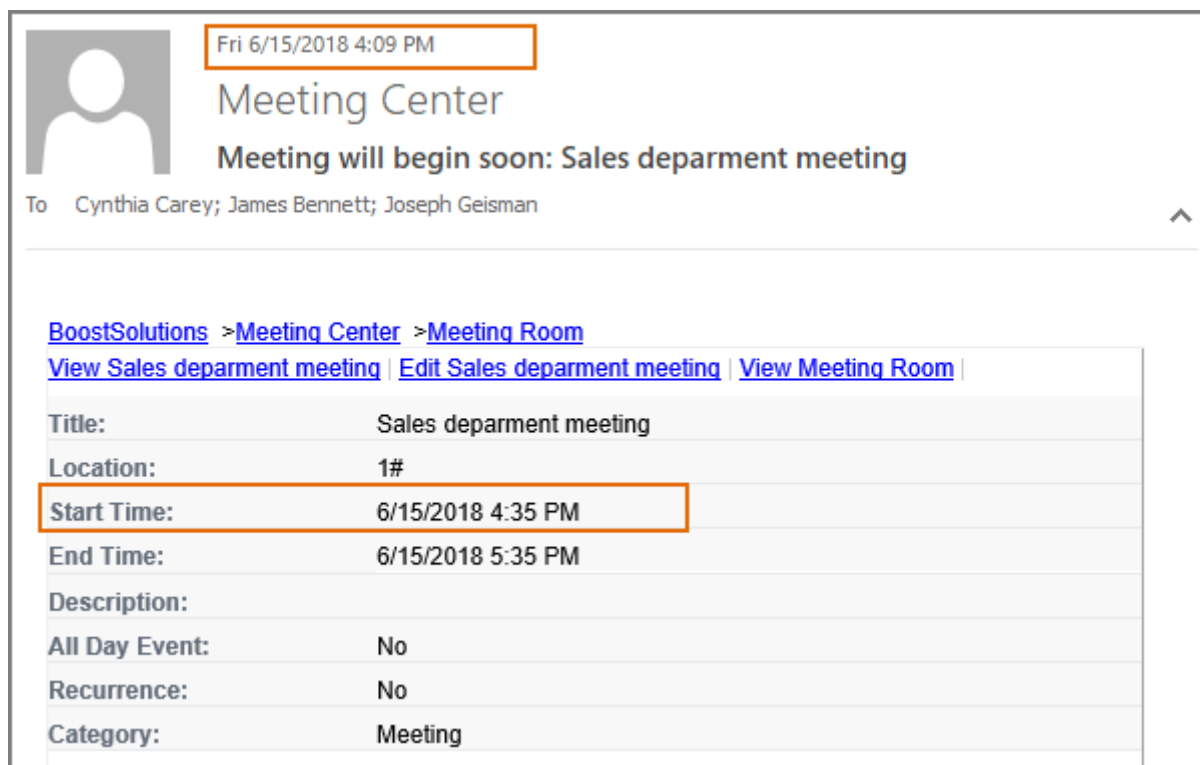


Figura 3 – Elementele unei notificări „push” [14]

Notificările prin email sunt acele mesaje trimise către utilizatori pentru a transmite informații, actualizări sau alerte legate de anumite evenimente.

Pentru promovarea unei afaceri, notificarea utilizatorilor prin email este cea mai ieftină modalitate. Există și alte avantaje în utilizarea acestui tip de notificări, precum plaja largă de utilizatori către care poate ajunge informația simultan sau accesibilitatea oferită în vremurile actuale, prin faptul că orice utilizator poate avea acces în permanență la email prin intermediul dispozitivelor mobile. Emailurile permit adăugarea a mai mult conținut într-un singur mesaj și pot fi mult mai personalizate, de exemplu se pot adăuga imagini sau chiar și HTML și CSS pentru a crea șabloane de newsletter [13].

Un avantaj semnificativ pentru aplicația „PawsPlan” este posibilitatea de a păstra notificările primite prin email, ceea ce întărește ideea de date centralizate într-un loc, cu șanse mici de a se pierde sau de a fi șterse din greșeală. Din moment ce notificările primite prin email din „PawsPlan” nu necesită o acțiune imediată, aceasta este abordarea potrivită pentru înștiințarea utilizatorilor.



The screenshot shows an email notification interface. At the top left is a placeholder icon for a person. To its right, the date and time 'Fri 6/15/2018 4:09 PM' are displayed in a box. Below this, the text 'Meeting Center' is followed by 'Meeting will begin soon: Sales deparment meeting' (note the typo). The recipients are listed as 'To Cynthia Carey; James Bennett; Joseph Geisman'. Below the header, there are navigation links: 'BoostSolutions > Meeting Center > Meeting Room', 'View Sales deparment meeting' (typo), 'Edit Sales deparment meeting' (typo), and 'View Meeting Room'. A table follows with meeting details:

Title:	Sales deparment meeting
Location:	1#
Start Time:	6/15/2018 4:35 PM
End Time:	6/15/2018 5:35 PM
Description:	
All Day Event:	No
Recurrence:	No
Category:	Meeting

Figura 4 – Exemplu de notificare prin email, de tip reminder [15]

3. TEHNOLOGII UTILIZATE

3.1. HTML

HTML (abreviere a HyperText Markup Language) este, așa cum îi spune numele, un limbaj de markup. Acesta contribuie la structurarea unor instrucțiuni specifice, cu ajutorul unor taguri, pentru a afișa conținutul unei pagini în World Wide Web. Cu alte cuvinte, cu ajutorul HTML se creează scheletul unei pagini web.

Alte limbajele de markup sunt XML, XHTML sau SVG, fiind esențiale pentru a prezenta documente și date în diferite contexte, precum stocarea datelor sau crearea graficelor.

HTML oferă creatorilor de aplicații un limbaj cunoscut universal, de către orice motor de căutare, prin care aceștia pot publica online documente cu text, tabele, imagini etc., dar pot și accesa informații deja existente pe internet și să le includă în aplicațiile proprii [3].

Scalabilitatea și extensibilitatea sunt două trăsături definitorii pentru acest limbaj. Dezvoltatorii pot crea atât pagini simple și fără complexitate, cât și aplicații complexe, de dimensiuni mai mari. Extensibilitatea este demonstrată prin folosirea atributelor personalizate, ce permit adăugarea de noi funcționalități în aplicații.

Comunitatea dezvoltatorilor de aplicații web este în continuă expansiune, iar aceștia descoperă constant noi moduri prin care pot îmbunătăți procesul de creare de aplicații, corespunzătoare pentru cât mai multe moduri diferite de lucru. Un plus care vine odată cu HTML 4.0 este în sfera accesibilității, deoarece include noi moduri prin care informația poate ajunge și la persoane cu nevoi speciale, precum nevăzătorii [3].

Prima pagină web a fost creată în 1991 de către Tim Bernes Lee, folosind doar limbajul HTML pur [7].

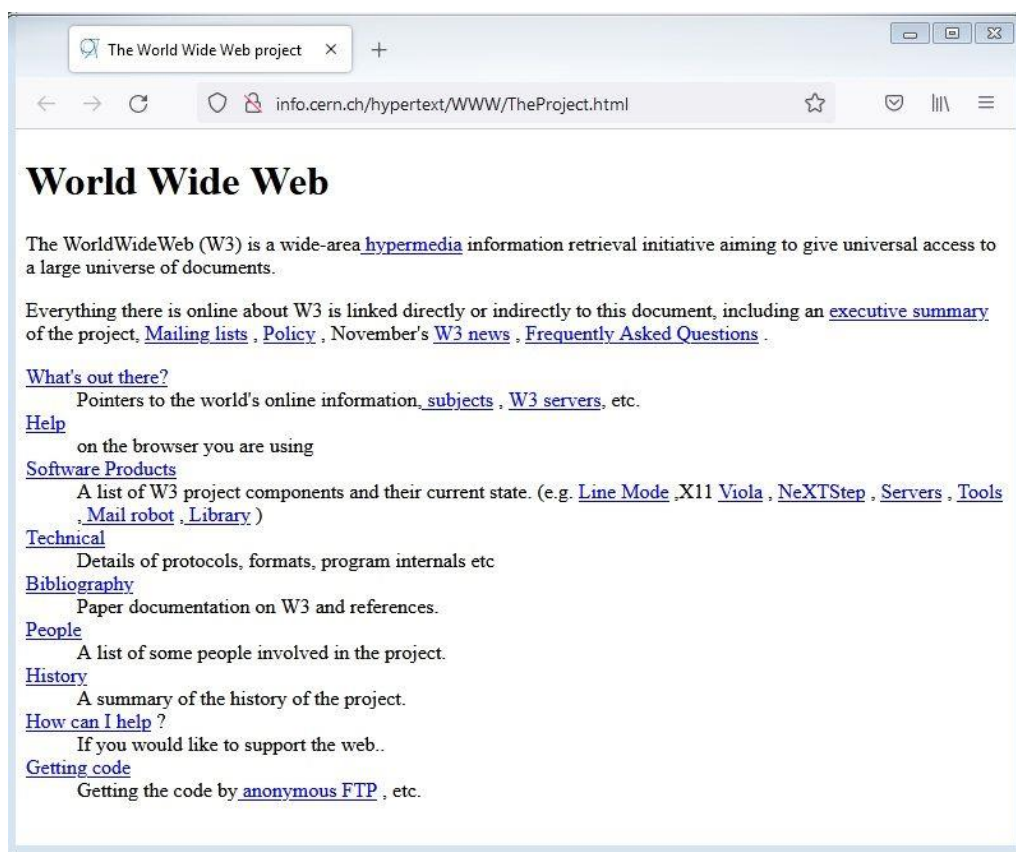


Figura 5 – Prima pagină web, „The WWW Project”, creată doar cu limbajul HTML [4]

3.2. CSS

În timp ce HTML are rolul de a structura informația într-o pagină web, CSS (abreviere a Cascading Style Sheets) contribuie la aspectul acesteia, cu scopul de a avea o imagine cât mai atractivă pentru utilizatori.

Dacă HTML poate crea o pagină web fără CSS, cel de-al doilea nu are o utilizare de sine stătătoare, nu se pot crea pagini web doar folosind CSS.

Codul în limbajul CSS poate fi scris atât în cadrul fișierelor HTML, cât și în fișiere separate, dedicate elementelor de stilizare. Cea de a doua modalitate contribuie la un cod cât mai curat și inteligibil, dar și la utilizarea unor stiluri în mai multe locuri, fără a avea componente redundante de cod. De asemenea, actualizarea se poate face mai ușor în acest caz. Totuși, dacă este necesar un număr mic de atribute, sau chiar unul singur, se poate folosi și abordarea în linie cu codul HTML.

CSS joacă un rol important în vizibilitatea aplicațiilor, deoarece utilizatorii le folosesc de pe dispozitive tot mai variate. Odată cu tranziția înspre telefoane mobile și tablete, web designerii trebuie să adapteze înfățișarea paginilor, pentru ca utilizatorii să nu sesizeze diferențe majore între utilizarea pe calculator sau pe dispozitive mobile [6].



Figura 6 – Folosirea media queries, parte a CSS3, pentru adaptarea la ecrane [8]

3.3. JAVASCRIPT

Cu tehnologiile introduse până acum, putem crea pagini web care sunt atât funcționale, cât și atractive din punct de vedere vizual, însă nu își pot schimba starea fără a declanșa reîmprospătarea paginii de către utilizator.

Din această nevoie de a crea aplicații în care utilizatorul să nu fie nevoit să reîmprospăteze pagina după anumite acțiuni, în anul 1995 a fost introdus limbajul JavaScript [1].

Acest limbaj este cunoscut în prezent de majoritatea motoarelor de căutare. Contribuie la modernizarea aplicațiilor web, ceea ce permite ca paginile să se reîmprospăteze singure atunci când este necesar, fără intervenția forțată a utilizatorului. Poate fi folosit și la crearea aplicațiilor tradiționale, venind cu un plus de interactivitate [1].

Flexibilitatea este o trăsătură caracteristică a acestui limbaj, care vine cu avantaje și dezavantaje. Avantajul principal este că permite aplicarea unor tehnici care, în alte limbaje nu sunt permise. Există și un dezavantaj în acest lucru, deoarece, pentru începători este mult mai greu să își descopere greșelile, nefiind semnalizate de către limbaj [1].

JavaScript și toate adaptările sale continuă să rămână în topul preferințelor programatorilor, conform „Stack Overflow Developer Survey” din 2022. JavaScript rămâne pe primul loc pentru al zecelea an la rând, ceea ce dovedește o dezvoltare constantă și benefică pentru programatori. Acest sondaj a avut 71.547 răspunsuri [9].

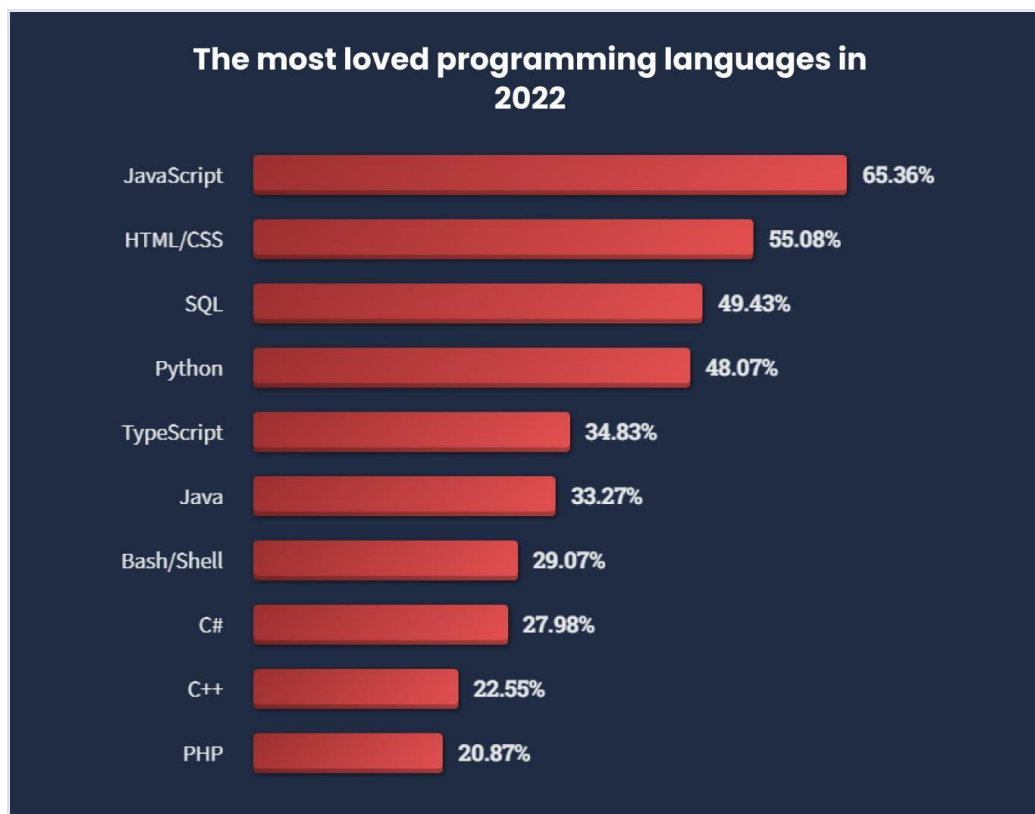


Figura 7 – Cele mai iubite limbaje de programare, conform „Stack Overflow Developer Survey”, din 2022

3.4. REACTJS

ReactJS este o bibliotecă open-source (se va folosi termenul „framework”) a limbajului JavaScript, folosită în front-end, în special pentru a construi interfețe cu utilizatorul (UI). Ceea ce face Reactul special este că permite crearea de componente reutilizabile, a căror stare din cadrul aplicației poate fi controlată cu ușurință [4].

Se poate numi bibliotecă a limbajului JavaScript deoarece se folosește pentru a crea aplicații JavaScript și permite omiterea scrierii anumitor porțiuni de cod, precum scripturi, și folosirea doar a porțiunilor de cod necesare direct în programele proprii [12].

Apariția acestui limbaj își are începuturile în anul 2010, când Facebook a introdus XHP (variante proprie de PHP) în tehnologiile folosite, fiind open-source. XHP permite crearea de componente, ceea ce face ca sintaxa să fie ulterior introdusă în React. În 2011 apare FaxJS, un prototip al cunoscutului React. În anul 2012 apare framework-ul React, pentru ca apoi, în 2013 să fie lansat în cadrul unei conferințe ca fiind o bibliotecă open-source [10].

Conform unui studiu realizat de medium.com, React rămâne constant în topul preferințelor dezvoltatorilor de aplicații web, pe mai multe planuri precum: disponibilitatea locurilor de muncă, căutările și descărcările pe internet sau satisfacția utilizatorilor [11].

Front-End Jobs by Framework

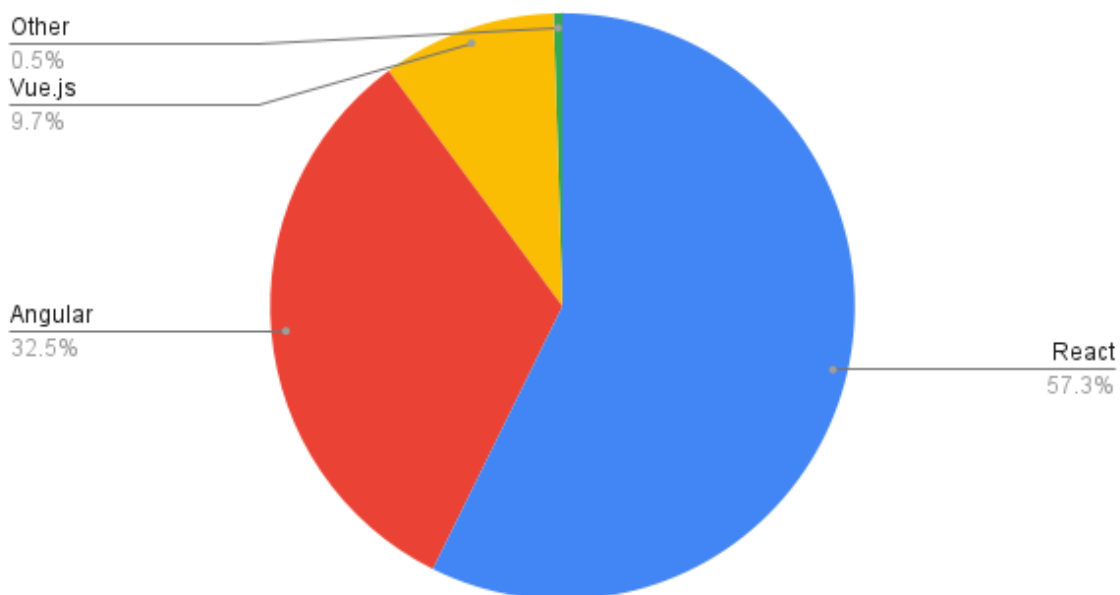


Figura 8 – Menționarea frameworkurilor de front-end în descrierile job-urilor [11]

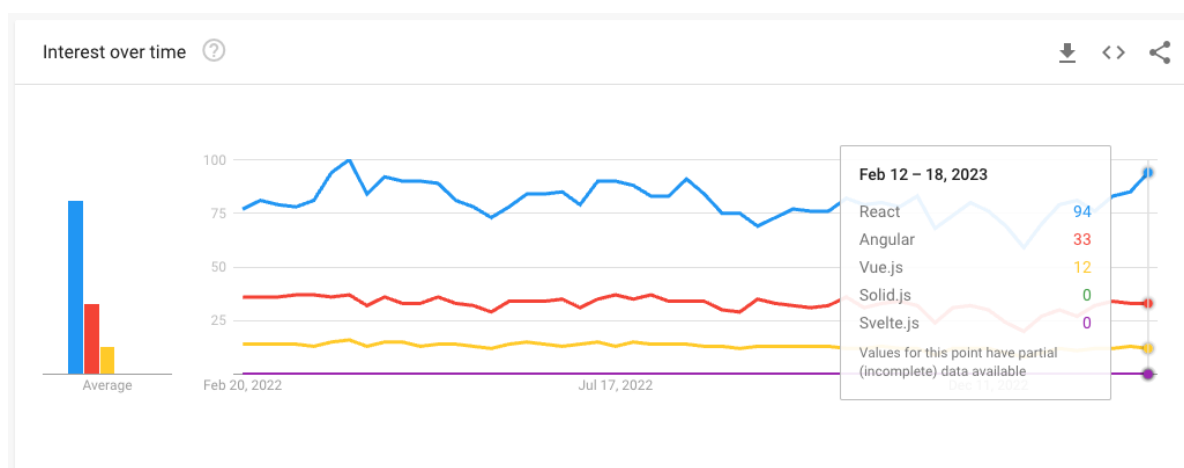


Figura 9 – Căutărilor de frameworkuri de front-end în 2022 [11]

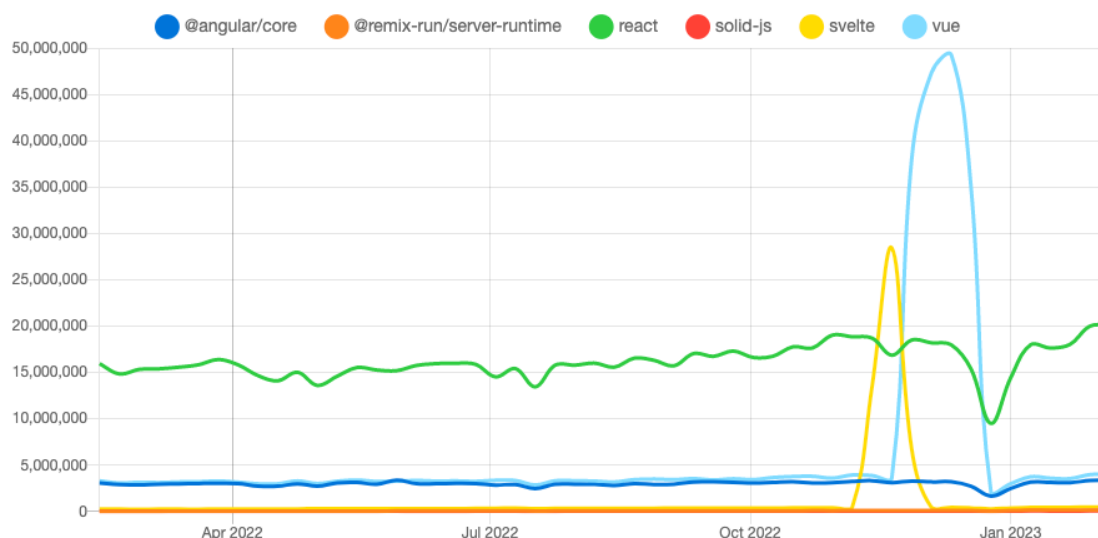


Figura 10 – Descărcările de frameworkuri de front-end în 2022 [11]

Satisfaction (Would Use Again)

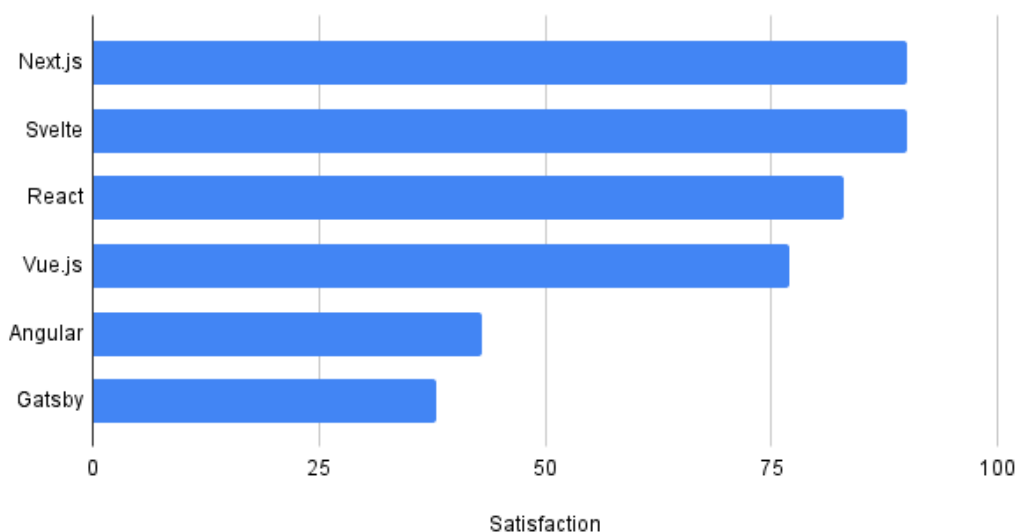


Figura 11 – Satisfacția utilizatorilor pentru frameworkuri de front-end în 2022 [11]

Cele mai populare biblioteci folosite în React sunt:

- React Router: permite crearea mai multor pagini și navigarea fără reîmprospătarea aplicației;
- Redux: contribuie la managementul stării aplicației, stocând stările intermediare și fluxurile de date;
- Axios: permite realizarea de requesturi HTTP, inclusiv requesturi asincrone către API (Application Programming Interface);

- Material-UI: oferă componente de UI prestabilite, pentru a ajuta programatorii să creeze aplicații atractive visual și funcționale;
- Formik: este o bibliotecă ce simplifică crearea de formulare din aplicațiile web;
- React Testing Library: bibliotecă cu ajutorul căreia se pot testa componentele React;
- React Spring: contribuie la crearea de animații pentru un UI dinamic;

3.5. FIREBASE

Firebase este o platformă ce a fost dezvoltată de Google și vine în ajutorul creatorilor de aplicații de orice fel (Android, iOS și web) pentru a construi și dezvolta proiectele dorite [17].

A crea o aplicație bună de la zero poate fi dificil, deoarece există numeroși factori care pot defini o aplicație ca fiind „bună”, de la gestionarea datelor primite și afișate în baze de date la aspecte ce țin de înștiințarea utilizatorilor cu privire la anumite acțiuni sau evenimente. Toți acești factori pot crește considerabil timpul și efortul necesar pentru a crea o aplicație și, mai mult, a o pune la dispoziție utilizatorilor la o scară largă [2].

Google a sesizat aceste nevoi ale creatorilor de aplicații și, astfel, a lansat în anul 2016 Firebase, platforma ce oferă instrumentele și mediul în care se pot construi aplicații, care apoi să se transforme într-o afacere de succes și să aducă venituri considerabile [2].

Printre serviciile oferite de Firebase se numără procesarea de plăți, metode de stocare a datelor, metode de autentificare și diferite analize personalizate în funcție de aplicația dezvoltată [18].

Datorită tuturor acestor aspecte, platforma Firebase este cunoscută de publicul larg de dezvoltatori ca fiind un înlocuitor pentru serverul unei aplicații. Totuși, recomandarea specialiștilor este ca aceasta să nu fie folosită ca un înlocuitor pentru construirea aplicațiilor Android, iOS sau web, ci, mai degrabă, ca o îmbunătățire cu servicii comune, sub umbrela Google, ce permite concentrarea mai mare pe implementarea unor idei creative, cu o interfață cu utilizatorul care să atragă și să fie accesibilă de folosit.

Firebase oferă o interfață accesibilă și intuitivă chiar și pentru developer, pentru început apărând în Firebase Console toate proiectele realizate. După selectarea unui anumit proiect, sunt disponibile serviciile utilizate pentru acesta. Platforma oferă trei categorii de servicii: pentru construirea aplicației, pentru lansare și monitorizare și pentru a atrage utilizatorii.

Serviciile pentru „Build-ul” aplicațiilor sunt cele care alcătuiesc back-endul aplicației și sunt următoarele:

- Realtime Database
- Cloud Firestore
- Remote Config
- Firebase Extensions
- App Check
- Cloud Functions
- Authentication

- Cloud Messaging
- Hosting
- Cloud Storage
- Firebase ML [19]

Serviciile de „Release & Monitor” sunt cele care ajută la testarea aplicației, dar și la lansarea anumitor funcționalități și putem aminti următoarele:

- Crashlytics
- Google Analytics
- Remote Config
- Performance Monitoring
- Test Lab
- App Distribution [19]

Serviciile de „Engage” sunt cele care contribuie la a menține interesul utilizatorilor asupra aplicației, fiind următoarele:

- Remote Config
- Google Analytics
- A/B Testing
- Authentication
- Cloud Messaging
- Crashlytics
- In-App Messaging

3.5.1. FIREBASE AUTHENTICATION

Un factor foarte important atât în crearea, cât și în utilizarea unei aplicații este securitatea informațiilor. Un prim pas în a controla cine are acces la ce informație îl reprezintă controlul utilizatorilor înregistrați și anume autentificarea. Serviciul oferit de Firebase pentru această funcționalitate este Firebase Authentication.

Acest serviciu vine cu funcționalități pentru implementarea logicii aplicației și kituri de dezvoltare software ușor de folosit, dar și cu librării cu elemente de interfață cu utilizatorul. Există o gamă variată de modalități de autentificare prin Firebase Authentication, de la un simplu email și parolă, la contul de GitHub [18].

Pentru a putea identifica conturile create de utilizatori, se generează un ID unic (UID), care este stocat, împreună cu alte date legate de autentificare (modul în care aceasta s-a realizat, adresa de email, data autentificării) în baza de date generată.

Pentru aplicația „PawsPlan” se folosește autentificarea cu email și parolă.

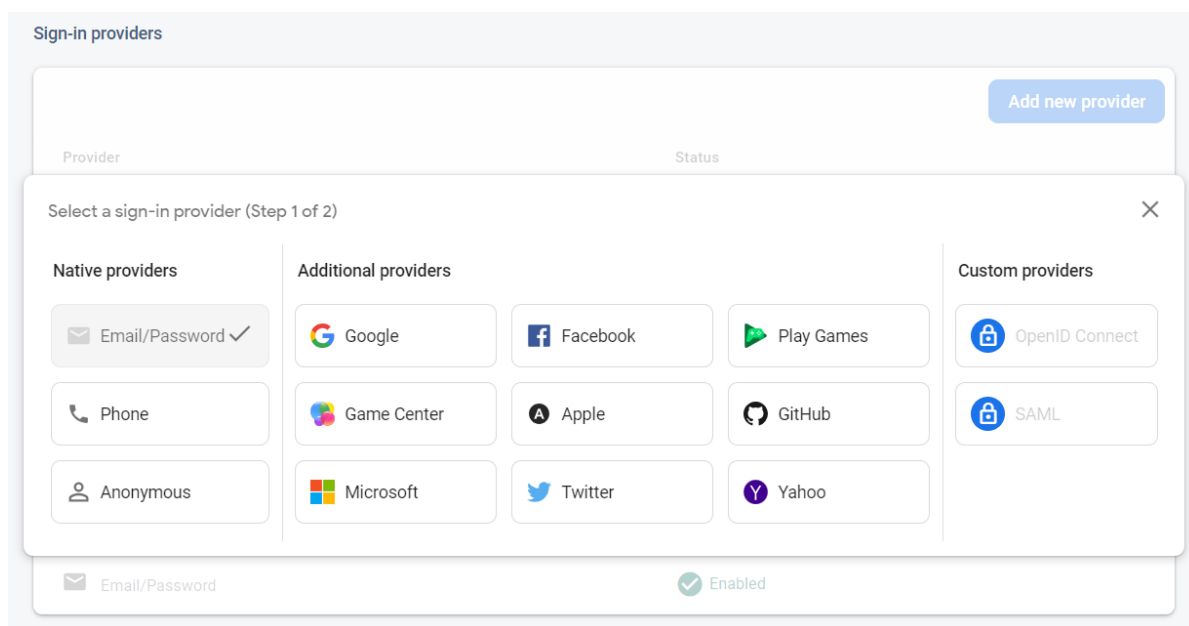


Figura 12 – Modalități de autentificare disponibile în Firebase Authentication

3.5.2. CLOUD FIRESTORE

Cloud Firestore este un serviciu al Firebase ce constă într-o bază de date de tip NoSQL. Este o bază de date flexibilă și scalabilă, disponibilă prin intermediul Google Cloud Storage, unde încarcă și actualizează constant datele pe care le primește. Cloud Firestore oferă suport chiar și fără conexiune la internet, deci problemele ce apar din cauza conexiunii slabe dispar [18].

Firebase oferă un kit de dezvoltare software, ce permite creatorilor de aplicații să aibă acces facil la datele folosite. Fiind o bază de date NoSQL, datele sunt stocate în documente, care, grupate, formează colecții. O colecție poate conține atât documente, cât și alte colecții imbricate.

Aplicația „PawsPlan” se folosește de serviciile Cloud Firestore pentru a stoca date suplimentare despre utilizatori, dar și date despre fiecare animal înregistrat și tratamentele acestuia. Adicional, există și date despre medicii veterinari înregistrați.

3.5.3. FIREBASE HOSTING

Firebase Hosting este un serviciu din suita Firebase, ce facilitează găzduirea și distribuirea de aplicații web. Este caracterizat prin simplitatea cu care se poate folosi, prin oferirea accesului la site-ul găzduit din orice colț al lumii, dar și prin integrarea armonioasă cu celelalte servicii Firebase.

Un aspect important în alegerea acestui serviciu este securitatea încorporată, deoarece oferă suport pentru HTTPS, asigurând o conexiune criptată între utilizator și server.

3.6. WEBSTORM

Pentru a crea aplicația „PawsPlan” am folosit IDE-ul (abreviere a Integrated Development Environment) numit WebStorm. Acesta este un mediu de dezvoltare creat de JetBrains special pentru dezvoltarea de aplicații web în JavaScript, având diverse ustensile ce fac scrierea codului mai facilă pentru dezvoltatori.

Printre funcționalitățile care m-au ajutat la crearea aplicației se numără:

- Asistența inteligentă pentru codare: auto-completarea codului, detectarea erorilor;
- Integrare cu alte platforme: de exemplu cu sisteme de versionare, precum Git, permite efectuarea operațiilor direct din IDE;
- Suport pentru framework-uri: asistența inteligentă nu se aplică doar limbajului JavaScript pur, ci și frameworkului React, folosit în aplicație.

3.7. VERSIONAREA CODULUI. GIT

Git este un sistem open-source de versionare a codului, ce ajută la urmărirea tuturor modificărilor aduse unui proiect software. Ajutorul cel mai mare pe care îl aduce dezvoltatorilor este că facilitează munca în echipă, iar aceștia pot lucra simultan în cadrul aceluiași proiect. Git oferă posibilitatea de a reveni la versiuni anterioare ale proiectului, rulând atât local, pe dispozitivele dezvoltatorilor, cât și online, prin intermediul platformelor precum GitHub, GitLab sau BitBucket [20].

Versionarea codului se poate face prin crearea de ramuri, oferind flexibilitate dezvoltatorilor. Ramurile (terminologia în limba engleză este mai folosită în domeniu, „branch”) oferă posibilitatea de a adăuga noi funcționalități unui proiect deja existent, fără a afecta stadiul deja salvat [20].

Pentru a gestiona operațiile de versionare a codului se pot folosi atât instrucțiuni în linia de comandă, cât și aplicații ce oferă interfețe grafice, precum GitKraken sau GitHub Desktop (folosit și de mine la crearea aplicației „PawsPlan”). Aceste aplicații facilitează urmărirea modificărilor, ce se pot face cu doar câteva clicuri.

3.8. BIBLIOTECI ALE REACTJS

Pentru îmbunătățirea performanțelor, atât în stadiul de dezvoltare a aplicației, cât și în cel de folosire a acesteia, ReactJS se poate folosi de biblioteci. Acestea sunt module sau pachete predefinite, create de alți dezvoltatori pentru a adăuga funcționalități comune, specifice proiectelor React.

Scopul acestora este de a contribui la reutilizarea codului, înglobând funcționalități precum manipularea datelor, gestionarea stării aplicației sau chiar componente de interfață cu utilizatorul.

În spatele tuturor acestor biblioteci există o comunitate activă de dezvoltatori, care oferă suport, documentație și contribuie la întreținerea și îmbunătățirea lor.

3.8.1. CREATE-REACT-APP

Această bibliotecă oferă punctul de start pentru o aplicație React, construind structura proiectului și având instrumentele necesare pentru a crea, testa și lansa o aplicație. Codul acesteia este disponibil pe GitHub și, conform aceleiași platforme, este folosită de peste 10 milioane de utilizatori.

3.8.2. REACT-ROUTER-DOM

React Router este o librărie populară printre utilizatorii React, ce permite implementarea funcționalității de a naviga printre „paginile” aplicației.

Chiar dacă aplicațiile React sunt formate dintr-o singură pagină, folosirea acestei librării face ca, pentru utilizator, să pară mai multe pagini, deoarece, în funcție de URL-ul accesat, sunt disponibile doar anumite componente din aplicație.

React-router-dom este pachetul special pentru aplicații web al React Router.

3.8.2 REACT-FONTAWESOME

Font Awesome nu este o librărie specifică React, dar, începând cu versiunea 5.15.4, are oficial o componentă specifică acestui framework [21].

Această librărie contribuie la stilizarea interfeței cu utilizatorul, punând la dispoziție icoane din orice domeniu, planul gratuit oferind o gamă variată de icoane și nefiind foarte restrictiv.

3.8.3 SWEETALERT

SweetAlert este o librărie a limbajului JavaScript, ce pune la dispoziție creatorilor de aplicații interfețe personalizabile și atractive de atenționare a utilizatorilor cu privire la o acțiune viitoare. De exemplu, dacă utilizatorul vrea să șteargă o înregistrare din baza de date, acesta va fi alertat înainte cu un mesaj de tipul „Ești sigur că vrei să ștergi înregistrarea?”.

3.8.4 EMAILJS

EmailJS este o bibliotecă pentru aplicațiile React, de tip SMTP (abreviere a Simple Mail Transfer Protocol), care facilitează trimiterea și primirea de emailuri, fără a fi nevoie de un server.

Avantajul cu care vine protocolul folosit de EmailJS este că previne trimiterea sau primirea de emailuri malițioase, iar platforma din motorul de căutare vine cu posibilitatea de a crea un șablon de email.

4. PROIECTAREA APLICAȚIEI

4.1. FUNCȚIONALITĂȚILE APLICAȚIEI

4.1.1. ÎNREGISTRAREA UTILIZATORULUI

„Ca și utilizator, vreau să îmi pot crea un cont în aplicație.”

Această funcționalitate face să se creeze o nouă intrare în baza de date pentru un utilizator care nu are deja un cont creat. Pagina conține un formular în care utilizatorul introduce informațiile:

- Nume
- Adresă de email
- Parolă

Când butonul *ÎNREGISTRARE* este apăsător, se creează o nouă intrare în baza de date.

4.1.2. AUTENTIFICAREA UTILIZATORULUI

„Ca și utilizator, vreau să mă pot conecta cu un cont deja creat.”

Utilizatorii care au deja un cont creat se pot autentifica prin intermediul unui formular, unde trebuie să introducă adresa de email și parola. Odată apăsător butonul *CONECTARE*, se verifică dacă utilizatorul există în baza de date, iar, în caz afirmativ, se realizează autentificarea.

4.1.3. DECONECTAREA UTILIZATORULUI

„Ca și utilizator, vreau să mă pot deconecta din aplicație.”

La apăsarea butonului *DECONECTARE*, utilizatorul este deconectat și nu mai are acces la aplicație până la o eventuală autentificare.

4.1.4. VIZUALIZAREA LISTEI DE ANIMALE

„Ca și utilizator, vreau să am acces la lista cu animalele mele.”

Dacă utilizatorul nu are animale adăugate, va primi un mesaj care îi spune să facă acest lucru. Dacă utilizatorul are animale adăugate în aplicație, va avea la dispoziție o listă simplificată, ce conține doar numele animalelor adăugate.

Odată ce selectează numele unui animal, se vor afișa detalii referitoare la acesta: numele, vârsta și rasa, dar și o listă cu medicațiile asigurate.

4.1.5 ADĂUGAREA UNUI ANIMAL

„Ca și utilizator, vreau să pot adăuga un nou animal în lista mea.”

Pentru a adăuga un animal în lista sa, utilizatorul trebuie să completeze într-un formular numele, vârsta și rasa animalului. Atunci când apasă butonul *ADAUGĂ ANIMAL*, se creează o nouă intrare în baza de date specifică.

4.1.6. EDITAREA UNUI ANIMAL

„Ca și utilizator, vreau să pot edita informațiile despre un animal deja adăugat în lista mea.”

Din pagina cu detalii despre animal, utilizatorul poate alege să editeze datele acestuia, prin butonul *EDITARE*. Astfel este direcționat către un formular precompletat cu datele inițiale ale animalului, pe care le poate modifica și apoi salva, prin butonul *SALVEAZĂ*.

4.1.7. ȘTERGEREA UNUI ANIMAL

„Ca și utilizator, vreau să pot șterge un animal din lista mea.”

Analog butonului de editare, există un buton *ȘTERGERE*, iar la apăsarea acestuia, utilizatorul poate elimina un animal din lista sa.

4.1.8. ADĂUGAREA UNEI MEDICAȚII

„Ca și utilizator, vreau să pot adăuga o medicație pentru un anumit animal din lista mea.”

Din pagina cu lista de medicații ale unui animal, utilizatorul poate adăuga o nouă medicație, prin butonul +. Astfel este direcționat către un formular, în care trebuie să introducă date referitoare la numele medicației, dozajul și data la care a fost administrată. Prin apăsarea butonului *ADAUGĂ*, se creează o nouă intrare în baza de date specifică.

4.1.9. ȘTERGEREA UNEI MEDICAȚII

„Ca și utilizator, vreau să pot șterge o medicație a unui animal din lista mea.”

În lista de medicații, în dreptul fiecărui element, există un o icoană ce reprezintă un coș de gunoi. Prin apăsarea acesteia, utilizatorul poate șterge o medicație din lista corespunzătoare unui animal.

4.1.10. LISTA VETERINARILOR PARTENERI

„Ca și utilizator, vreau să pot vizualiza lista cu veterinarii existenți în aplicație.”

Apăsând butonul *CABINETE VETERINARE*, utilizatorul are acces la lista cabinetelor partenere, având disponibile date despre acestea, precum: nume, adresă și număr de telefon.

4.1.11. TRIMITEREA NOTIFICĂRIILOR PRIN EMAIL

„Ca și manager, vreau să pot declanșa automat trimiterea de notificări prin email către utilizatori.”

Aplicația are o pagină accesibilă doar de către contul cu acest rol, prin care, cu doar un buton, poate trimite emailurile către utilizatori.

4.2. DIAGRAMA UML A CAZURILOR DE UTILIZARE

Pentru a proiecta sistemul aplicației „PawsPlan” a fost necesară gândirea unor cazuri de utilizare care pot fi întâlnite de către utilizatori.

Principalele scenarii de navigare în aplicație sunt enumerate în paragraful 4.1 și pot fi vizualizate schematic în figura de mai jos.

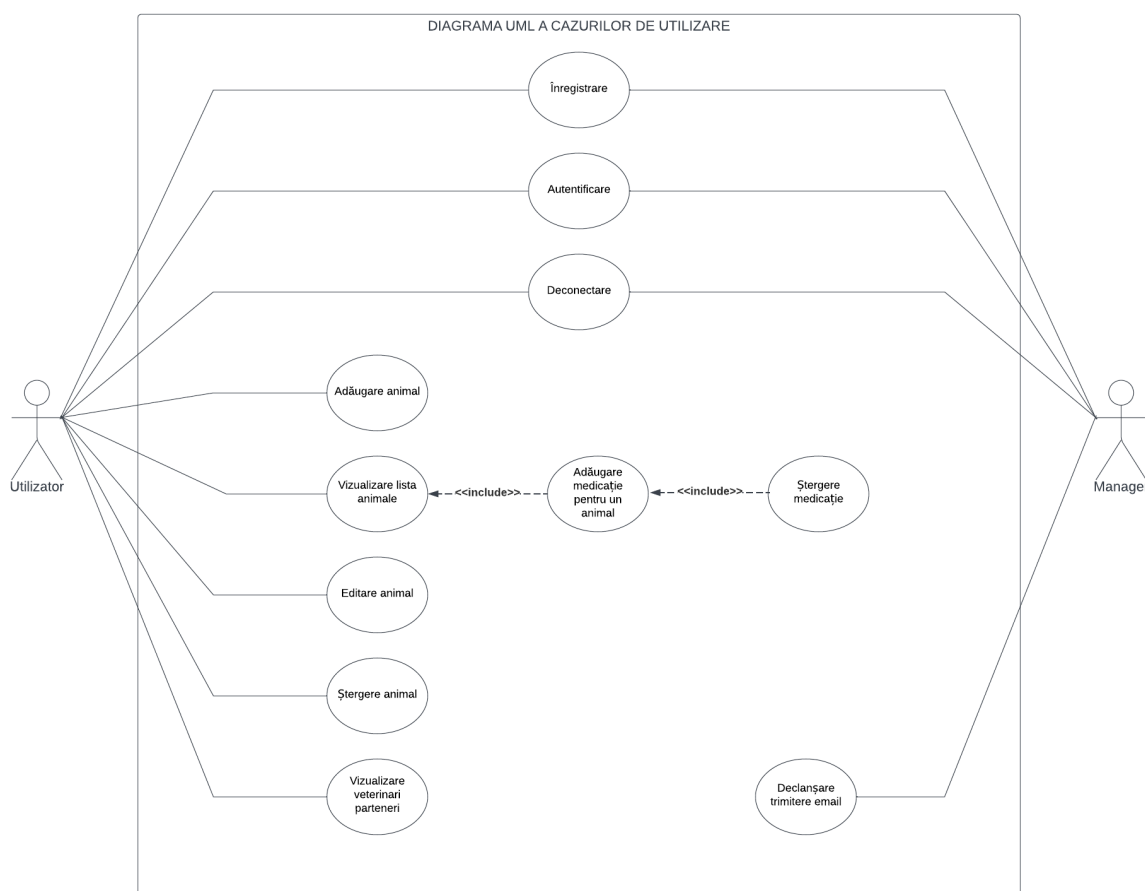


Figura 13 – Diagrama UML a cazurilor de utilizare a aplicației PawsPlan

4.3. DIAGRAMA DE FLUX AL UNUI UTILIZATOR

Un alt tip de diagramă ce m-a ajutat în faza de proiectare a aplicației este diagrama de flux. Aceasta descrie „ciclul de viață” al unui utilizator, de când accesează aplicația și până o părăsește, în cazul aplicației „PawsPlan” de la autentificare până la deconectare.

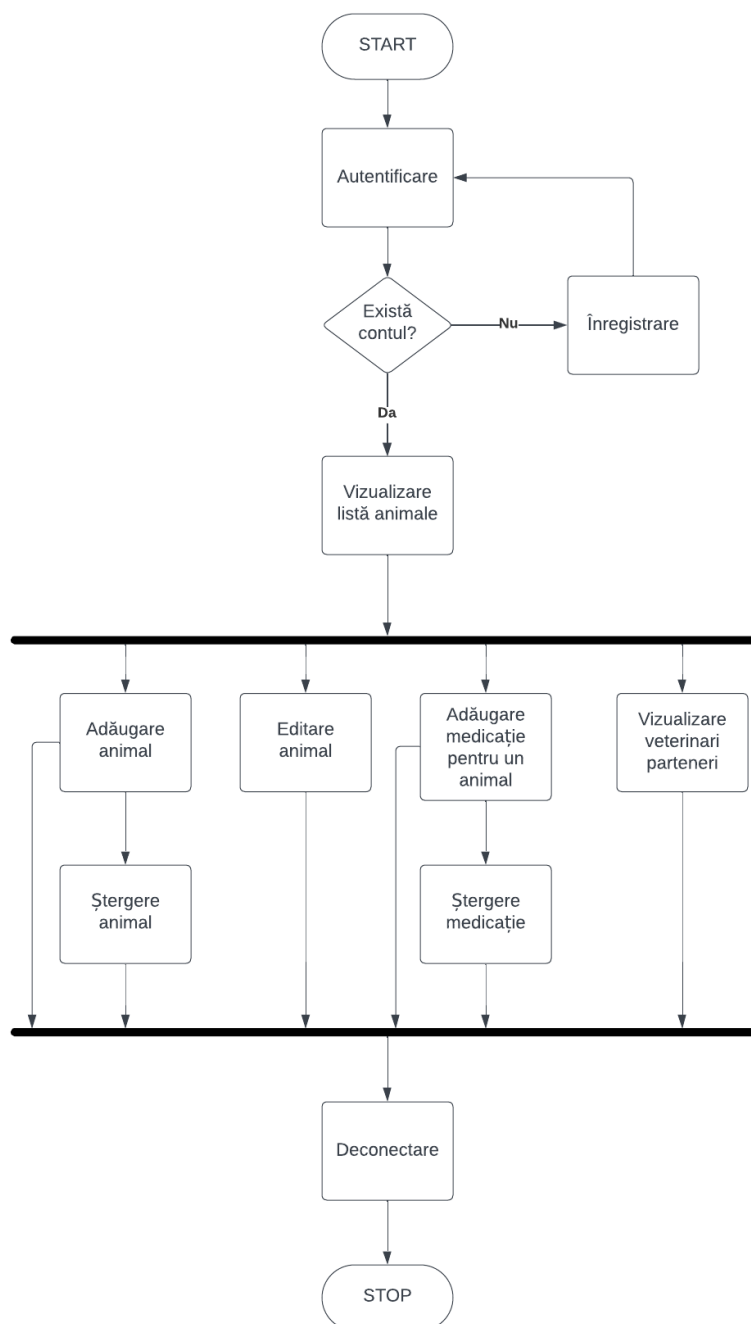


Figura 14 – Diagrama de flux al utilizatorului în aplicația PawsPlan

4.4. PROIECTAREA BAZEI DE DATE

Pentru a stoca informațiile folosite în aplicație, a fost necesară folosirea bazei de date Firebase Cloud Firestore. Aceasta este o bază de date de tip NoSQL, adică nu are datele stocate în tabele, ci în colecții și documente. O colecție poate conține documente sau, la rândul ei, o subcolecție, aceste structuri putând fi imbricate.

Pentru această aplicație am folosit două colecții principale: *users* (pentru a stoca datele despre utilizatori) și *vets* (pentru a stoca datele despre veterinarii parteneri).

Colecția de utilizatori conține documente care au același ID cu cel generat și stocat în urma folosirii sistemului Firebase Authentication, pentru a putea exista o legătură între acesta și Cloud Firestore. Spre deosebire de serviciul de autentificare, celălalt conține mai multe date referitoare la utilizatori.

Fiecare document asociat unui utilizator are câmpurile de nume (denumit *ownerName*), email și o subcolecție numită *pets*, în care sunt stocate datele despre animalele utilizatorului respectiv.

Colecția de animale conține date cu privire la numele (*petName*), vârsta (*age*) și rasa (*breed*) animalului, dar și o subcolecție ce conține date despre medicațiile asociate fiecărui animal.

Colecția de medicații, denumită *medications*, are câmpuri cu numele medicației (*medName*), dozajul (*dosage*) și ziua în care aceasta a fost administrată (*timestamp*).

La același nivel cu colecția de utilizatori există colecția de veterinari, cu câmpuri ce conțin informații despre numele cabinetului, adresa și numărul de telefon.

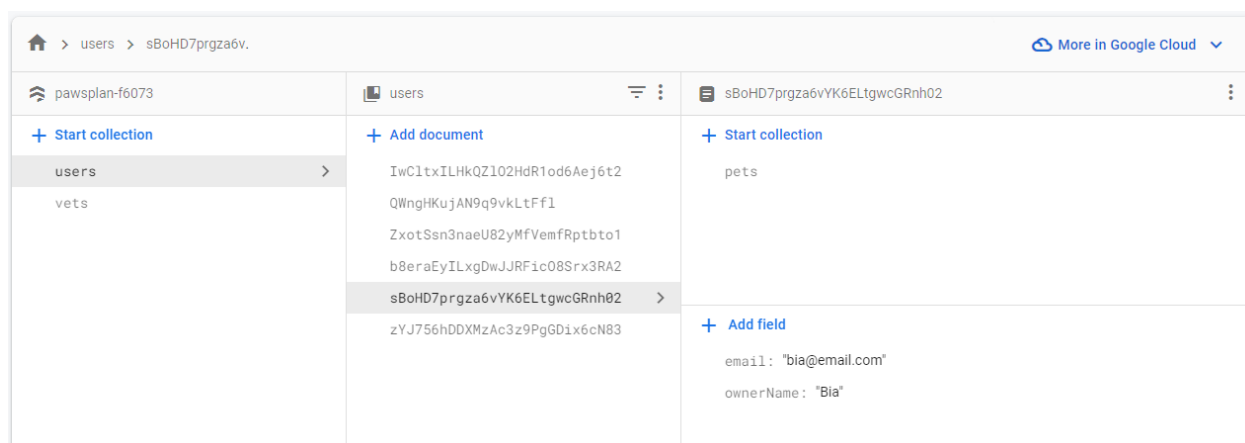


Figura 15 – Structura unui document aflat în colecția de utilizatori

Home > users > sBoHD7prgza6v... > pets > doRgy62QbZdD... More in Google Cloud		
sBoHD7prgza6vYK6ELtgwcGRnh02	pets	doRgy62QbZdDhVgPZW0Y
+ Start collection	+ Add document	+ Start collection
pets	AsfDd9GgFmpBs07KYuBF	medication
	doRgy62QbZdDhVgPZW0Y	
	tGtK5puSFw19hWE3wmf0	
+ Add field		+ Add field
email: "bla@email.com" ownerName: "Bla"		age: 3 breed: "Golden retriever" petName: "Snoopy"

Figura 16 – Structura unui document aflat în colecția de animale

Home > ... > pets > doRgy62QbZdD... > medication > UjB45p4xH9IRVD41jFHL More in Google Cloud		
doRgy62QbZdDhVgPZW0Y	medication	UjB45p4xH9IRVD41jFHL
+ Start collection	+ Add document	+ Start collection
medication	UjB45p4xH9IRVD41jFHL	+ Add field
	1rIfYUeJWIhViSkhi3K8	
+ Add field		dosage: "1" medName: "deparazitare externa" timestamp: "2023-06-22"
age: 3 breed: "Golden retriever" petName: "Snoopy"		

Figura 17 – Structura unui document aflat în colecția de medicații

Home > vets > R8WhBdkNVsIX... More in Google Cloud		
pawsplan-f6073	vets	R8WhBdkNVsIXGxW1Es6u
+ Start collection	+ Add document	+ Start collection
users	2Nks8fFr7GWHvY6FSZZo	+ Add field
vets	7ey1wLY6D1854FD5G2Q3	adresa: "Bd. Liviu Rebreanu Nr. 127"
	BUTMJH8ra7YCQrph4ZHV	cabinet: "Clinica Veterinară – Dr. Mihai Sărbu"
	0b1DHs186sz6YdOK39Mc	telefon: "0752129129"
	R8WhBdkNVsIXGxW1Es6u	
	kwcv78NIi7w7NsWjeZb3	
	zkuo9M5DX1MEVzmQWasQ	

Figura 18 – Structura unui document aflat în colecția de veterinari

5. IMPLEMENTARE

5.1. ÎNREGISTRAREA UTILIZATORULUI

Pentru înregistrarea unui nou utilizator, serviciul Firebase Authentication are niște funcții prestabilite, ce adaugă în baza de date o nouă intrare.

Formularul de înregistrare este completat de către utilizator, iar la apăsarea butonului *ÎNREGISTRARE*, se declanșează funcția *handleRegister*, în cadrul căreia se apelează funcția predefinită *createUserWithEmailAndPassword*, ce realizează înregistrarea.

Această componentă folosește librăriile React hooks ('useState') pentru a gestiona stările datelor din aplicație, React Router ('useNavigate') pentru a redirecționa utilizatorul către ecranul de autentificare după înregistrare, dar și kitul de dezvoltare software specific Firebase.

În cazul în care înregistrarea nu se poate realiza, se va afișa un mesaj de eroare.

```
const handleRegister = async (e) => {  
  e.preventDefault();  
  
  try {  
    const userCredential = await auth.createUserWithEmailAndPassword(email, password);  
    const user = userCredential.user;  
  
    const userRef = firebase.firestore().collection(collectionPath: 'users').doc(user.uid);  
    const userData = {  
      ownerName: name,  
      email: user.email  
    };  
    await userRef.set(userData);  
  
    setRegisteredEmail(email);  
    setEmail(value: '');  
    setPassword(value: '');  
  
    navigate('/login');  
  } catch (error) {  
    console.error('Eroare la înregistrarea utilizatorului:', error);  
  }  
};
```

Figura 19 – Înregistrarea utilizatorului în aplicație

5.2. AUTENTIFICAREA UTILIZATORULUI

Analog cu componenta de înregistrare, cea de autentificare preia datele introduse de utilizator în formular și declanșează funcția *handleLogin*. Aceasta verifică dacă există o înregistrare în baza de date cu credențialele introduse, iar dacă există, realizează autentificarea cu ajutorul funcției *signInWithEmailAndPassword*.

După autentificarea cu succes, datele sunt stocate în spațiul de stocare al sesiunii curente și pot și accesate din orice componentă a aplicației.

```
<div className="login-content">
  <div><h2>Conectare</h2></div>
  <form onSubmit={handleLogin} className="login-form">
    <div className="login-label">
      <FontAwesomeIcon icon={faEnvelope}/>
      <input
        className="login-input"
        type="email"
        placeholder="Email"
        value={email}
        onChange={(e : ChangeEvent<HTMLInputElement> ) => setEmail(e.target.value)}
      />
    </div>
    <div className="login-label">
      <FontAwesomeIcon icon={faLock}/>
      <input
        className="login-input"
        type="password"
        placeholder="Parola"
        value={password}
        onChange={(e : ChangeEvent<HTMLInputElement> ) => setPassword(e.target.value)}
      />
    </div>
    <button className="login-button" type="submit">CONECTARE</button>
  </form>
</div>
```

Figura 20 – Secvența de cod JSX ce conține formularul de autentificare

5.3. VIZUALIZAREA LISTEI DE ANIMALE

Pentru a implementa componenta de vizualizare a listei simplificate de animale ale unui utilizator am avut nevoie de un query ce ia datele necesare din Cloud Firestore și le adaugă într-o listă sub forma unui vector de obiecte.

Această componentă extrage datele și le transmite și către componentele ce conțin detalii despre animal și lista de medicații.

Lista simplificată conține doar numele animalelor, pe care le afișează separat, în câte un card.

În secvența de cod de mai jos, *useEffect* conține extragerea datelor din Cloud Firestore, bazat pe *userId*. Vectorul *petsData* ia valorile și le atribuie variabilei *pets*, prin intermediul stării corespunzătoare: *const [pets, setPets] = useState([])*;

```
useEffect( effect: () => {  
  const fetchPets = async () => {  
    try {  
      const petsQuery = query(collection(firestore, path: "users", userId, "pets"));  
      const petsSnapshot = await getDocs(petsQuery);  
      const petsData = petsSnapshot.docs.map((doc : QueryDocumentSnapshot<DocumentData> ) => {  
        const data = doc.data();  
        return {  
          petId: doc.id,  
          petName: data.petName,  
          age: data.age,  
          breed: data.breed  
        };  
      });  
      setPets(petsData);  
    } catch (error) {  
      console.error("Eroare la afișarea animalelor.", error);  
    }  
  }  
  fetchPets();  
}, deps: [userId]);
```

Figura 21 – Secvența de extragere a datelor din Firestore și crearea vectorului de animale

Pentru fiecare animal selectat, se modifică conținutul componentei *PetDetails*. Pentru a transmite proprietățile de la o componentă la alta, am folosit ca și *props* o variabilă numită *selectedPet*, ce conține ID-ul animalului selectat.

Astfel, am putut accesa proprietățile obiectului (nume, vârstă, rasă) și afișa în componentă, cu ajutorul JSX.

```
return (  
  <div className="info-pet">  
    <div className="info">  
      <h2>{selectedPet.petName}</h2>  
      <div className="info-card">  
        <p><FontAwesomeIcon className="icon" icon={faCalendarDays}/>Vârsta: {selectedPet.age}</p>  
        <p><FontAwesomeIcon className="icon" icon={faHeart}/>Rasa: {selectedPet.breed}</p>  
      </div>  
      <div className="animal-buttons">  
        <button onClick={handleEditPetRedirect}>Editare</button>  
        <button onClick={handleDelete}>Ștergere</button>  
      </div>  
    </div>  
  </div>  
)
```

Figura 22 – Accesarea proprietăților obiectului prin `{selectedPet.petName}`, `{selectedPet.age}`, `{selectedPet.breed}`

5.4. ADĂUGAREA UNUI ANIMAL

Componenta de adăugare a unui animal conține ca și cod JSX un formular, care, odată ce este trimis, declanșează funcția *handleAddPet*.

Se folosește React hook-ul *useState*, iar stările inițiale pentru câmpurile formularului sunt un șir de caractere gol pentru nume și rasă și 0 pentru vârstă.

Funcția *handleAddPet* conține o componentă externă, numită Sweet Alert, care constituie o validare adițională a operației de adăugare a unui animal. Odată ce butonul de confirmare al alertei este apăsă, se apelează funcția *addPet*, care adaugă în baza de date valorile introduse în formular și acum stocate în variabilele de stare. După ce intrarea este adăugată cu succes în baza de date, stările revin la valoarea inițială și se produce redirectionarea către pagina principală.


```
const handleAddPet = async (e) => {  
  e.preventDefault();  
  if (petName !== '') {  
    Swal.fire({ options: {  
      title: 'Adaugă animal',  
      text: 'Ești sigur că vrei să adaugi animalul?',  
      icon: 'question',  
      showCancelButton: true,  
      confirmButtonColor: '#180026FF',  
      cancelButtonColor: '#8B0000FF',  
      confirmButtonText: 'Adaugă',  
      cancelButtonText: 'Anulare'  
    })}.then((result : SweetAlertResult<Awaited<any>> ) => {  
      if (result.isConfirmed) {  
        addPet();  
      }  
    }));  
  }  
}
```

Figura 23 – Secvența de cod ce conține Sweet Alert-ul de confirmare a operației

```
const addPet = async () => {  
  try {  
    const userDocRef = doc(firestore, path: 'users', userId);  
    const petsCollectionRef = collection(userDocRef, path: 'pets');  
    const petDocRef = await addDoc(petsCollectionRef, data: {  
      age: age,  
      breed: breed,  
      petName: petName,  
    });  
  
    setPetName( value: '');  
    setAge( value: 0);  
    setBreed( value: '');  
  
    navigate('/');  
  } catch (error) {  
    console.error('Eroare la adaugarea animalului:', error);  
  }  
};
```

Figura 24 – Adăugarea animalului în baza de date și redirectionarea către pagina principală

5.5. EDITAREA UNUI ANIMAL

Componenta de editare a unui animal este asemănătoare cu cea de adăugare a unui animal. Diferențele se observă la modul în care sunt transmise și folosite datele referitoare la animalul ce urmează să fie editat.

În componenta cu detaliile animalului există butonul de *EDITARE*, care, apăsând, declanșează o funcție de redirectare, *handleEditPetRedirect*. Aceasta construiește o adresă URL formată din */edit-pet/?petId=*, la care concatenează ID-ul animalului respectiv. Pe lângă URL, la redirectare se transmite și starea actuală a proprietăților.

Stările proprietăților sunt primite de către componenta *EditPet* și folosite pentru a inițializa stările din aceasta. Aceste valori inițiale ale stărilor sunt afișate în formularul de editare și pot fi modificate sau nu. Și în acest caz se folosește o alertare a utilizatorului.

```
const handleEditPetRedirect = () => {  
  navigate(`/edit-pet?petId=${selectedPet.petId}`, {  
    state: {  
      petName: selectedPet.petName,  
      age: selectedPet.age,  
      breed: selectedPet.breed  
    }  
  });  
}
```

Figura 25 – Funcția de redirectare, ce construiește adresa URL dorită

5.6. ȘTERGEREA UNUI ANIMAL

Ștergerea unui animal se face din componenta *PetDetails*, cea care conține toate detaliile referitoare la animal. La apăsarea butonului *ȘTERGERE*, este apelată funcția *handleDelete*, ce declanșează alertarea prin funcția *.fire()*.

Dacă alertarea este confirmată, cu ajutorul funcției *deletePet*, se determină referința la înregistrarea dorită din baza de date, iar asupra acesteia se aplică funcția asincronă *deleteDoc*.

În cazul ștergerii efectuate cu succes, pagina este reîmprospătată, cu ajutorul instrucțiunii *window.location.reload()*.

5.7. VIZUALIZAREA MEDICAȚIILOR PENTRU UN ANIMAL

Această funcționalitate este disponibilă datorită a două componente: *MedForPet* și componenta sa părinte, *ViewPets*.

Cea din urmă conține mai multe sub-componente către care transmite date prin intermediul *props* și printre care se numără și componenta de afișare a medicațiilor pentru un animal selectat.

Stocarea listei de medicații se face într-un vector, ce inițial este gol. Odată ce se selectează un animal, ID-ul acestuia este transmis către componenta *MedForPet*, dar, pe lângă aceasta, este folosit și în componenta curentă, în React hook-ul *useEffect*.

Acest hook este dependent de modificarea valorii câmpului *selectedPet*, transmis ca și *props*. De fiecare dată când se modifică valoarea, deci ID-ul selectat, se declanșează funcția *fetchMedications*. Aceasta face o interogare în baza de date pentru a extrage datele din colecția *users*, pentru un *userId* specific, din colecția *pets* a acestui *userId*, pentru un *petId* specific, din colecția *medications*. Apoi, adaugă fiecare intrare în vectorul menționat anterior.

Aceasta este cea mai „în adâncime” interogare făcută în baza de date din proiectul Cloud Firestore al aplicației „PawsPlan”.

```
useEffect( effect: () => {  
  const fetchMedications = async () => {  
    try {  
      if (selectedPet) {  
        const medicationsRef = collection(firestore, {path: 'users', userId, 'pets', selectedPet.petId, 'medication'});  
        const medicationsQuery = query(medicationsRef);  
        const medicationsSnapshot = await getDocs(medicationsQuery);  
        const medicationsData = medicationsSnapshot.docs.map((doc : QueryDocumentSnapshot<DocumentData> ) => {  
          const data = doc.data();  
          return {  
            medId: doc.id,  
            medName: data.medName,  
            dosage: data.dosage,  
            timestamp: data.timestamp,  
          };  
        });  
        setMedications(medicationsData);  
      }  
    } catch (error) {  
      console.error('Eroare la afișarea medicatiilor!', error);  
    }  
  };  
  fetchMedications();  
}, deps: [userId, selectedPet]);
```

Figura 26 – Interogarea în baza de date pentru a crea lista de medicații

Pentru a afișa fiecare medicație în câte un card, am transmis către componenta *MedForPet*, prin intermediul *props*, și vectorul de medicații format anterior. Proprietățile obiectelor din acest vector sunt accesate și afișate prin secvența de cod JSX de mai jos, de exemplu prin `{medication.medName}`.

```
return (
  <div className="meds">
    <div className="header">
      <h2>Medicația lui {selectedPet.petName}</h2>
      <button className="addmed"
        onClick={handleAddMedication}>
        <FontAwesomeIcon className="icon" icon={faPlus}/>
      </button>
    </div>
    <div className="med-pets">
      {medications.map((medication) => (
        <div key={medication.medId} className="med-card">
          <p><strong>Nume:</strong> {medication.medName}</p>
          <p><strong>Dozaj:</strong> {medication.dosage}</p>
          <p><strong>Administrare:</strong> {medication.timestamp && medication.timestamp.toString()}</p>
          <button className="deletemed" onClick={
            () => handleDeleteMedication(medication.medId)}>
            <FontAwesomeIcon className="icon" icon={faTrash}/>
          </button>
        </div>
      ))}
    </div>
  </div>
)
```

Figura 27 – Afișarea listei de medicații pentru un animal selectat

5.8. ADĂUGAREA UNEI MEDICAȚII PENTRU UN ANIMAL

Componenta *MedForPet* menționată anterior, conține un buton cu simbolul +. La apăsarea acestuia, se apelează funcția *handleAddMedication*, care, la fel ca și funcționalitatea de editare a unui animal, formează o adresă URL din */add-medication?petId=*, la care concatenează valoarea câmpului *petId* pentru animalul selectat și realizează redirectionarea către acest URL, ce conține componenta *AddMedication*.

Această componentă reprezintă un nou formular, cu câmpurile *medName*, de tipul *string*, *dosage*, de tipul *number* și *timestamp*, de tipul *timestamp*. Toate acestea sunt tipuri de date predefinite în *Firebase Cloud Firestore*.

După adăugarea cu succes a înregistrării în baza de date, cele trei stări corespunzătoare celor trei câmpuri din formular sunt actualizate la valori nule.

```
const addMedication = async () => {
  try {
    const medication = {
      medName: medName,
      dosage: dosage,
      timestamp: timestamp
    };

    const petRef = collection(firestore, { path: 'users', userId, 'pets', petId, 'medication' });
    await addDoc(petRef, medication);

    setMedName({ value: '' });
    setDosage({ value: '' });
    setTimestamp({ value: '' });

    navigate('/');
  } catch (error) {
    console.error('Error adding medication:', error);
  }
};
```

Figura 28 – Secvența de cod de adăugare a unei medicații specifice unui animal

5.9. ȘTERGEREA UNEI MEDICAȚII PENTRU UN ANIMAL

Ștergerea unei medicații se realizează analog cu ștergerea unui animal, din cadrul componentei *MedForPet*. La apăsarea butonului cu simbolul coșului de gunoi corespunzător medicației care se dorește a fi ștearsă, este apelată funcția *handleDeleteMedication*, ce declanșează alertarea prin funcția *.fire()*.

Dacă alertarea este confirmată, cu ajutorul funcției *deleteMedication*, se determină referința la înregistrarea dorită din baza de date, iar asupra acesteia se aplică funcția asincronă *deleteDoc*.

În cazul ștergerii efectuate cu succes, pagina este reîmprospătată, cu ajutorul instrucțiunii *window.location.reload()*.

5.10. LISTA VETERINARILOR PARTENERI

Lista veterinarilor parteneri este o componentă asemănătoare cu cea ce conține lista de medicații pentru un animal.

Stocarea listei se face într-un vector, ce inițial este gol, dar urmează să fie populat printr-o interogare în baza de date, în colecția *vets*.

Conținutul vectorului se mapează, iar pentru fiecare intrare din colecția *vets*, se creează o componentă JSX, cu proprietăți extrase.

```
return (  
  <div className="body-container">  
    <Menu/>  
    <div className="content-container">  
      <div className="content">  
        <div className="content-vets">  
          {vets.map((vet) => (  
            <div className="vet-card">  
              <h2>{vet.cabinet}</h2>  
              <p> <FontAwesomeIcon className="icon" icon={faLocationDot}/>{vet.adresa}</p>  
              <p> <FontAwesomeIcon className="icon" icon={faPhone}/>{vet.telefon}</p>  
            </div>  
          ))}  
        </div>  
      </div>  
    </div>  
  </div>  
)
```

Figura 29 – Afișarea listei cu date despre veterinari, prin maparea vectorului

5.11.NOTIFICAREA PRIN EMAIL A UTILIZATORULUI

Notificarea prin email a utilizatorilor se face cu ajutorul serviciului EmailJS. Pentru a configura acest serviciu, a fost nevoie de crearea unui cont pe platformă, de selectarea furnizorilor de email și de crearea unui șablon de email, ce se va trimite utilizatorilor.

În cadrul aplicației React, am creat o nouă componentă, ce face o interogare în baza de date cu utilizatori și construiește un vector de obiecte cu numele utilizatorului și adresa acestuia de email. Avem nevoie email pentru a ști cui trimitem notificarea și de numele utilizatorului deoarece va apărea în conținutul emailului.

Trimiterea propriu-zisă a emailurilor se face prin intermediul funcției *sendEmail*, ce parcurge vectorul și apelează funcția *.send* pe fiecare obiect din vector.

```
const sendEmail = () => {  
  users.forEach((user) => {  
    const templateParams = {  
      to_name: user.ownerName,  
      to_email: user.email,  
    };  
  
    emailjs  
      .send(  
        serviceID: 'service_1',  
        templateID: 'template_1',  
        templateParams,  
        userID: 'user_1',  
      )  
      .then((result : EmailJSResponseStatus) => {  
        console.log('Email sent successfully to', user.user_email);  
      })  
      .catch((error) => {  
        console.error('Failed to send email to', user.user_email, error);  
      });  
  });  
};
```

Figura 30 – Funcția ce declanșează trimiterea emailurilor către utilizatori

The screenshot shows the EmailJS 'My Default Template' configuration interface. On the left, there's a sidebar with navigation links like 'Email Services', 'Email Templates', 'Contacts', etc. The main area is titled 'My Default Template' and has tabs for 'Content', 'Auto-Reply', 'Attachments', 'Contacts', and 'Settings'. The 'Content' tab is active, showing a rich text editor. The 'Subject' field contains 'Nu uita de animaluțu tău'. The 'Content' field contains 'Bună, {{{to_name}}}' and 'Nu uita de medicația animaluțu tău'. The 'To Email' field in the right sidebar contains '{{to_email}}'. Arrows point to these fields.

Figura 31 – Inserarea variabilelor `to_name` și `to_email` în șablonul de email din EmailJS

5.12. GĂZDUIREA ONLINE A APLICAȚIEI

Pentru ca aplicația să fie disponibilă nu doar local, ci și pe orice dispozitiv, indiferent de zona geografică, am ales să o distribuim printr-un serviciu ce permite găzduirea de aplicații online. Astfel, aplicația se găsește la adresa <https://pawsplan-f6073.web.app/>.

Folosind suita Firebase atât pentru autentificarea utilizatorilor, cât și pentru stocarea datelor în baza de date, am considerat că serviciul Firebase Hosting este cel potrivit pentru găzduirea aplicației „PawsPlan”.

Lansarea aplicației către publicul larg a fost posibilă cu doar câteva instrucțiuni în linia de comandă, cele mai importante fiind:

- *npm install -g firebase-tools* pentru a instala interfața în linie de comandă;
- *firebase init* pentru a inițializa și conecta aplicația cu Firebase;
- *npm run build* pentru a rula aplicația și a o pregăti de lansare;
- *firebase deploy* pentru a lansa aplicația pe serverele Firebase.

Un aspect important în procesul de găzduire a aplicației este conexiunea securizată dintre utilizator și server, garantată prin oferirea de către Firebase a suportului pentru HTTPS (HTTP securizat), protejând astfel aplicația de atacuri. Totodată, Firebase include automat certificatul SSL, verificând astfel identitatea serverului web.

6. UTILIZAREA APLICAȚIEI

Dacă utilizatorul nu este autentificat în aplicație, acesta are acces doar la prima pagină, ce conține butoanele de înregistrare și autentificare.

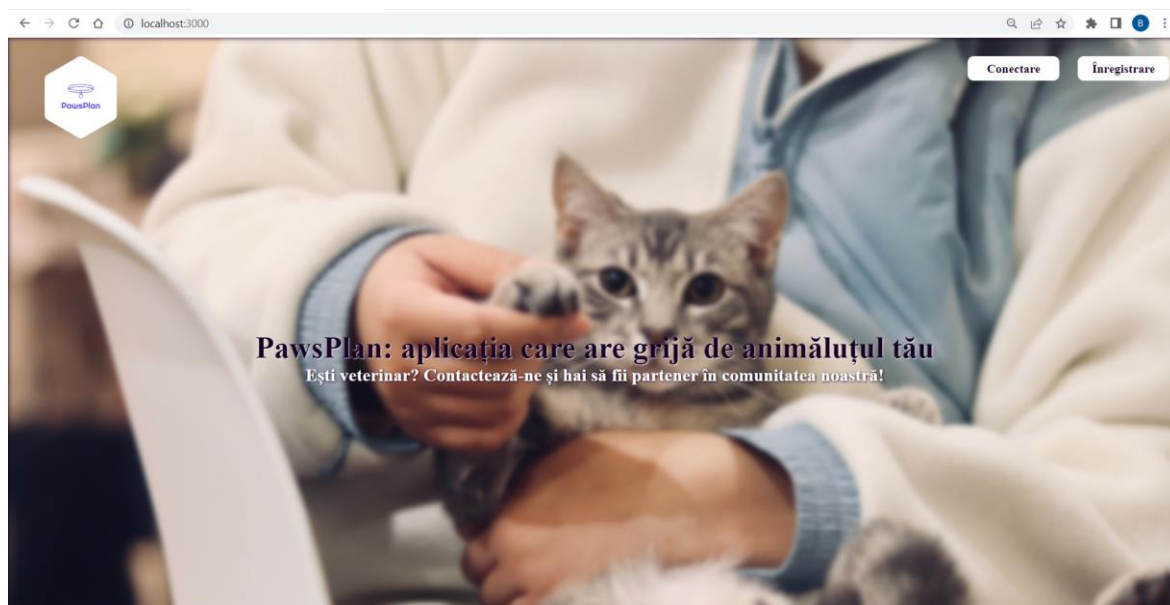


Figura 32 – Prima pagină, în cazul în care utilizatorul nu este autentificat

6.1. ÎNREGISTRAREA UTILIZATORULUI

Dacă utilizatorul nu are un cont creat, în cazul în care apasă pe butonul *ÎNREGISTRARE*, acesta este redirecționat către formularul de creare cont și se poate înregistra prin intermediul acestuia.

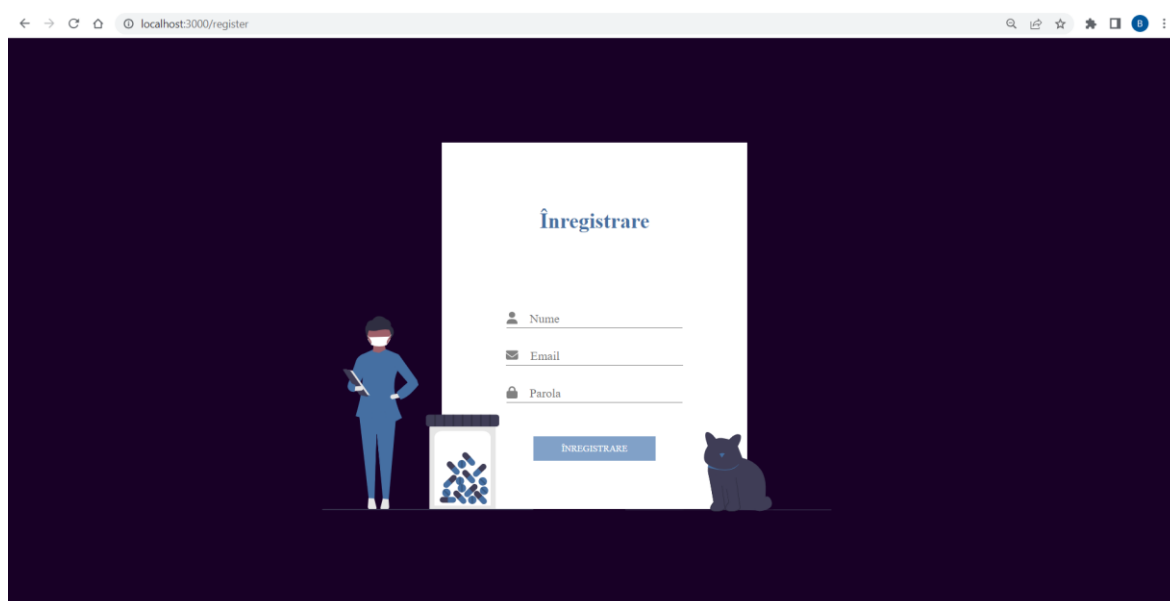


Figura 33 – Formularul de înregistrare

6.2. AUTENTIFICAREA UTILIZATORULUI

În cazul în care utilizatorul are deja un cont creat, îl poate accesa prin intermediul formularului de conectare.

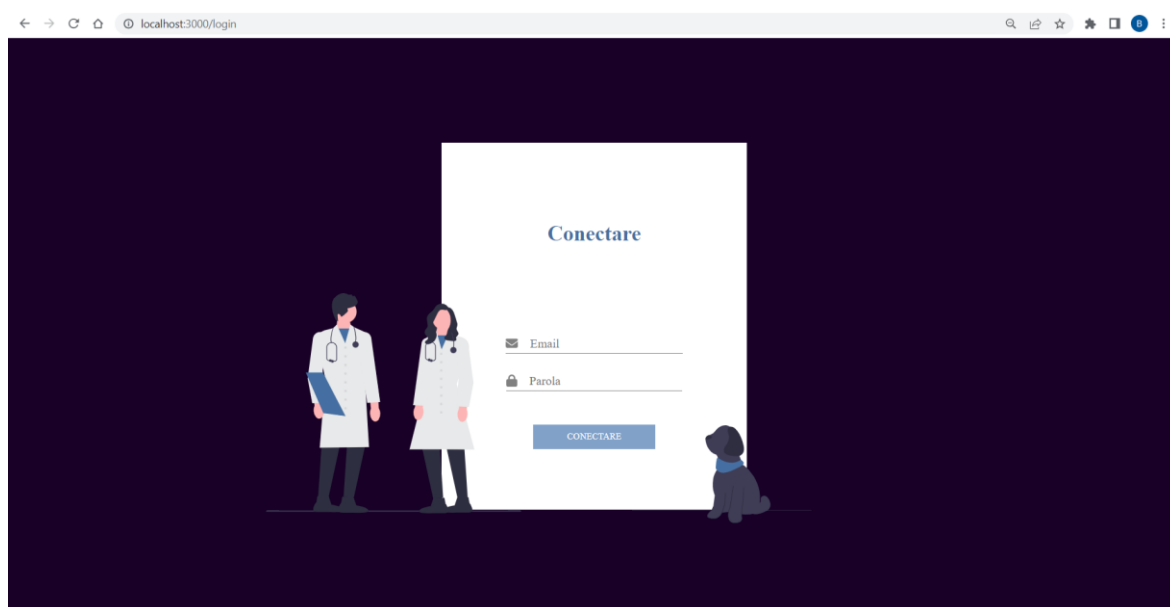


Figura 34 – Formularul de conectare

6.3. VIZUALIZAREA LISTEI DE ANIMALE

Odată ce utilizatorul este autentificat, poate accesa lista cu numele animalelor sale. Dacă selectează numele unui animal, în partea dreaptă vor apărea detaliile complete ale animalului respectiv.

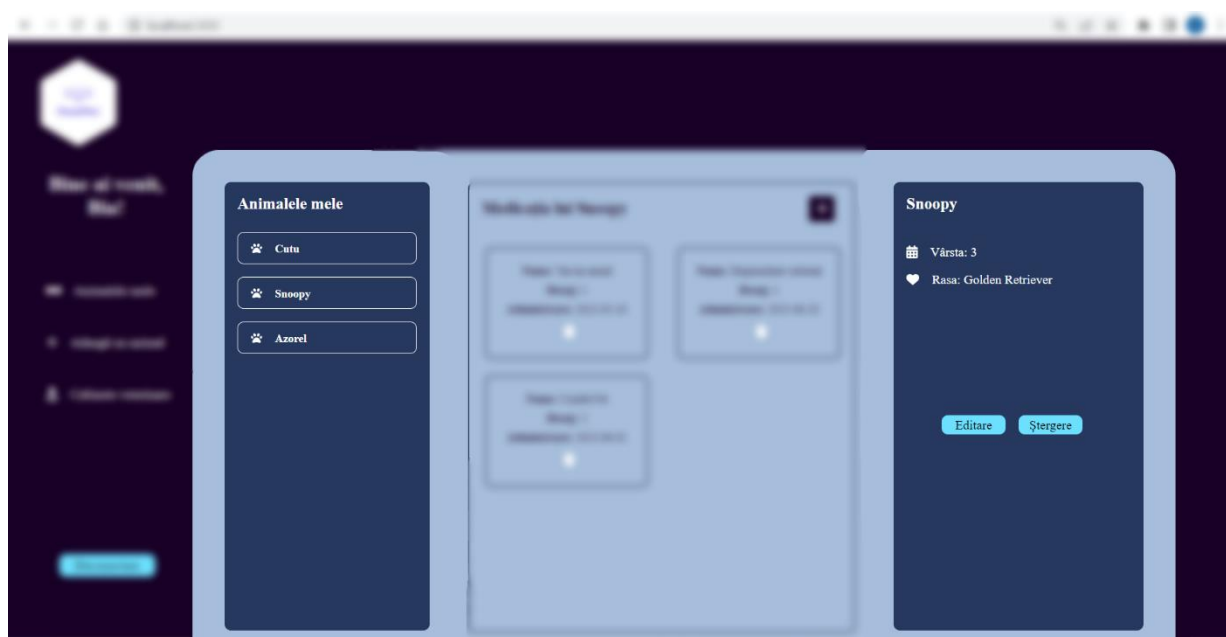
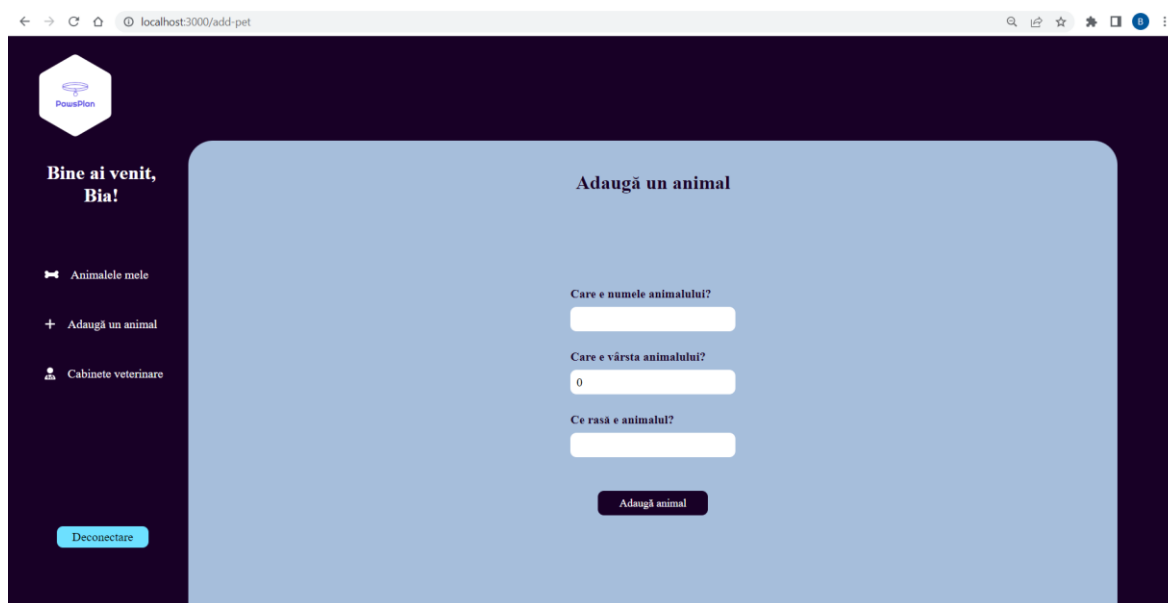


Figura 35 – Lista cu numele animalelor și detaliile animalului selectat

6.4. ADĂUGAREA UNUI ANIMAL

Din meniul din partea stângă, utilizatorul poate adăuga animale în lista sa, fiind redirectionat către formularul care face acest lucru.

Adăugarea nu se poate face fără alertarea preventivă a utilizatorului. După ce confirmă adăugarea prin alertă, este direcționat înapoi către pagina principală.



The screenshot shows a web browser window at localhost:3000/add-pet. The application has a dark purple sidebar on the left with a 'PawsPlan' logo and a welcome message 'Bine ai venit, Bia!'. The sidebar contains a menu with 'Animalele mele', 'Adaugă un animal' (highlighted with a plus icon), and 'Cabinete veterinare'. At the bottom of the sidebar is a 'Deconectare' button. The main content area is light blue and titled 'Adaugă un animal'. It contains three input fields: 'Care e numele animalului?', 'Care e vârsta animalului?' (with the value '0'), and 'Ce rasă e animalul?'. Below these fields is a dark purple button labeled 'Adaugă animal'.

Figura 36 – Formularul de adăugare a unui animal

6.5. EDITAREA UNUI ANIMAL

Formularul de adăugare a unui animal nu prezintă diferențe majore față de cel de adăugare a unui animal.

Pe lângă aspectul interfeței grafice, cea mai semnificativă diferență pentru utilizator este că, la accesarea acestui formular, câmpurile sunt deja completate cu valorile curente și așteaptă să fie actualizate.

The screenshot shows a web browser window with the URL `localhost:3000/edit-pet?petId=doRgy62QbZdDHVgPZW0Y`. The main content is a white modal box titled "Editează animal" (Edit animal) centered on a dark purple background. Inside the modal, there are three input fields: "Numele animalului:" (Animal name) with the value "Snoopy", "Vârsta animalului:" (Animal age) with the value "3", and "Rasa animalului:" (Animal breed) with the value "Golden Retriever". Below these fields is a blue button labeled "Salvează" (Save).

Figura 37 – Formularul de adăugare a unui animal

6.6. ȘTERGEREA UNUI ANIMAL

Pentru a putea șterge un animal, butonul care face acest lucru se găsește în componenta din partea dreaptă a aplicației, ce conține detaliile despre animal.

La apăsarea butonului **ȘTERGERE**, utilizatorul este înștiințat de această operație și se cere o aprobare suplimentară.

În cazul în care apasă pe butonul **ANULARE**, nu se întâmplă nimic cu înregistrarea animalului. În cazul în care apasă pe butonul **OK**, animalul dispăre, iar pagina este reîmprospătată automat.

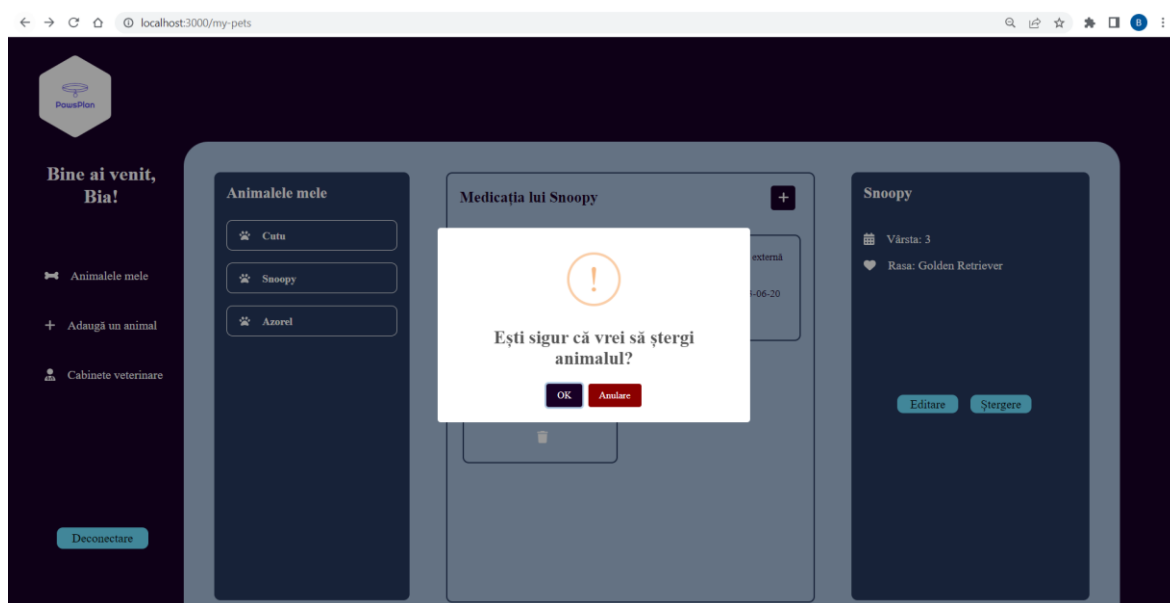


Figura 38 – Alertarea utilizatorului cu privire la ștergerea unui animal

6.7. VIZUALIZAREA MEDICAȚIILOR PENTRU UN ANIMAL

Această funcționalitate depinde de componenta din stânga ei, anume de lista simplificată a animalelor. Odată selectat numele unui animal din această listă, medicațiile lui apar într-o listă, cu toate detaliile.

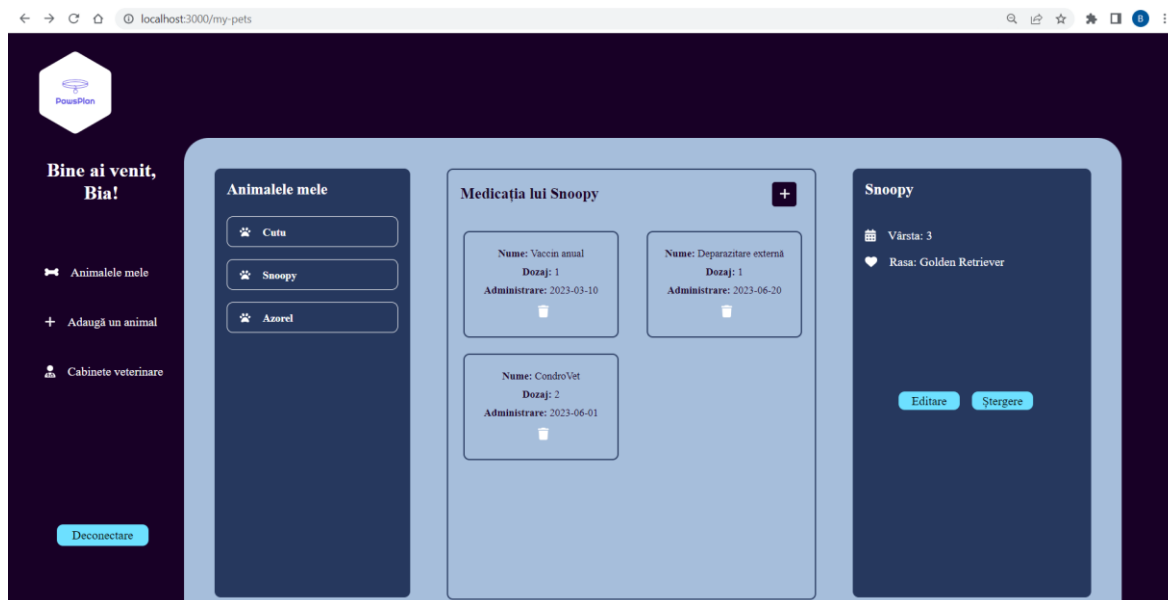


Figura 39 – Lista de medicații ale unui animal selectat

6.8. ADĂUGAREA UNEI MEDICAȚII PENTRU UN ANIMAL

În partea dreaptă sus a componentei cu lista de medicații pentru animalul selectat există un buton cu simbolul +, iar prin apăsarea acestuia, utilizatorul este redirecționat către pagina ce conține un formular de adăugare a unei medicații.

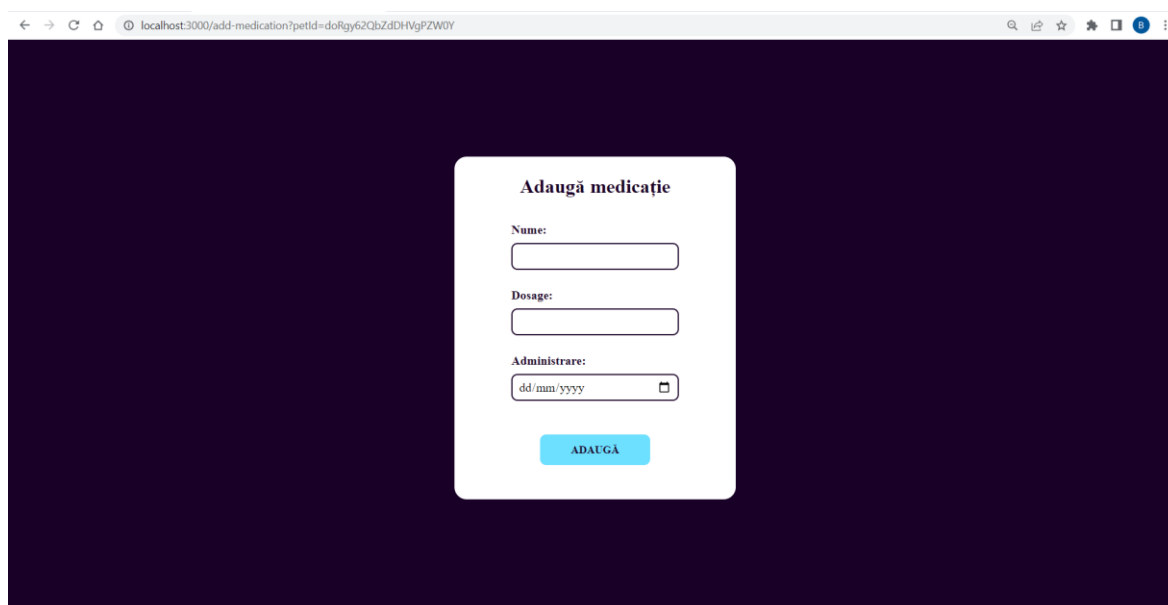


Figura 40 – Formularul de adăugare a unei medicații

După ce confirmă operația în cadrul alertei de înștiințare, utilizatorul este redirecționat către pagina principală.

6.9. ȘTERGEREA UNEI MEDICAȚII PENTRU UN ANIMAL

Fiecare card ce conține o medicație are în dreptul său un buton cu simbolul unui coș de gunoi. Dacă utilizatorul apasă pe butonul corespunzător medicației pe care dorește să o șteargă, va fi înștiințat printr-o alertă de operația ce urmează să o facă, putând să își confirme sau să își infirme intenția.

6.10. LISTA VETERINARILOR PARTENERI

Pe lângă toate funcționalitățile referitoare la gestiunea animalelor și a medicațiilor acestora, aplicația „PawsPlan” oferă și o pagină cu detalii despre unele cabinete veterinare, precum numele, adresa și numărul de telefon. Această pagină este disponibilă prin intermediul butonului *CABINETE VETERINARE*, din meniul aplicației.

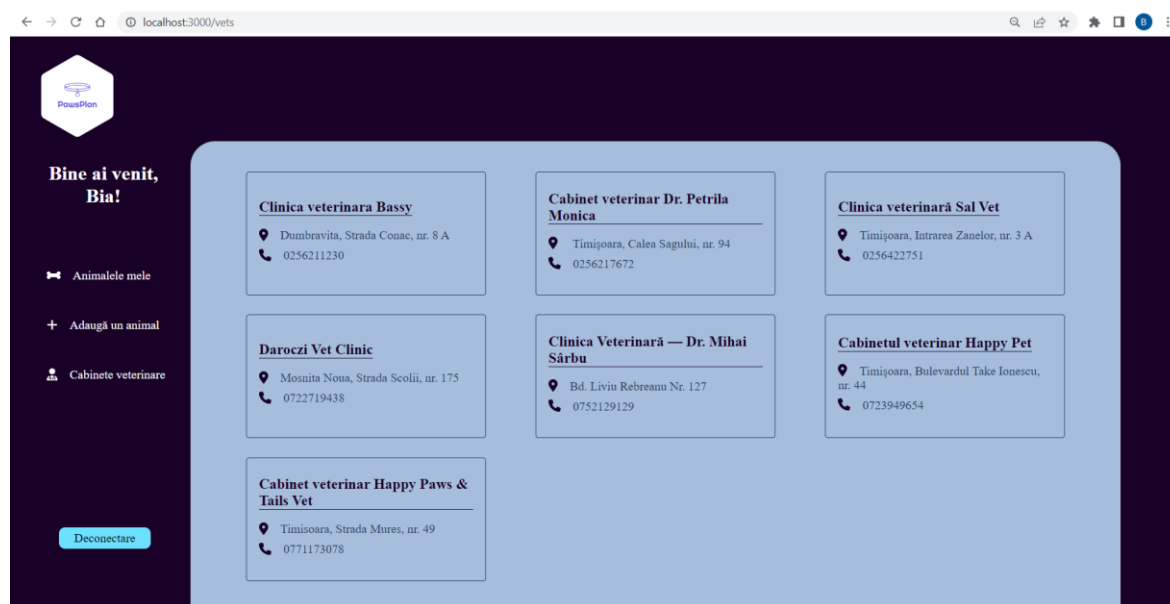


Figura 41 – Pagina cu detalii despre cabinetele veterinare partenere

6.11. NOTIFICAREA PRIN EMAIL A UTILIZATORULUI

Notificarea prin email a utilizatorilor are scopul de a aduce aminte de medicațiile pe care le au animalele.

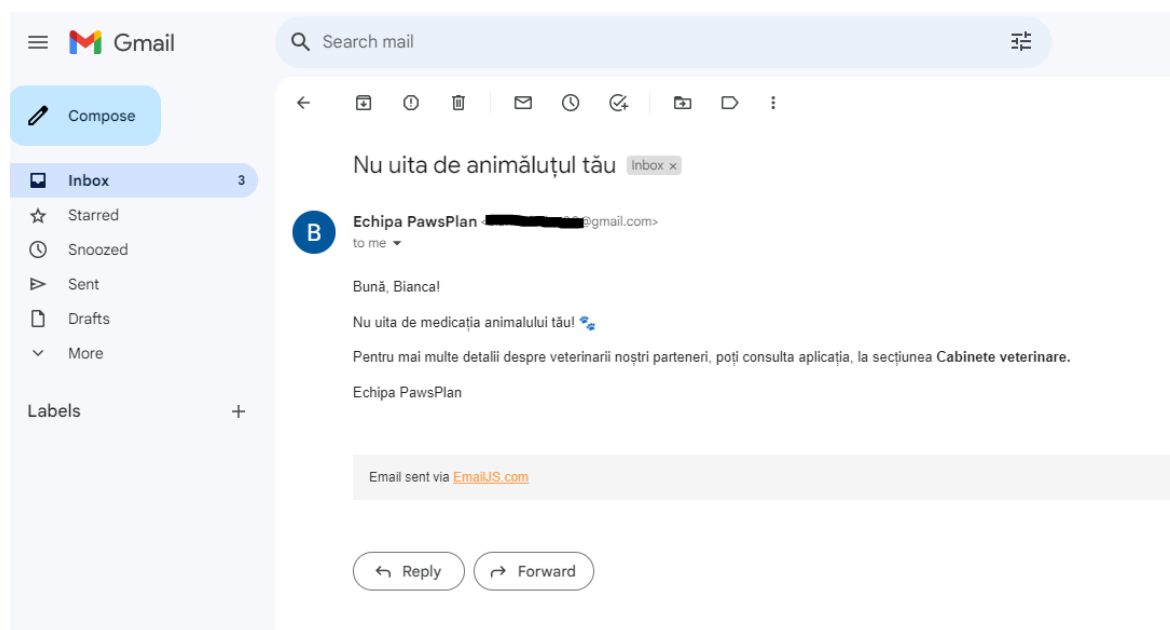


Figura 42 – Conținutul emailului trimis automat către utilizatori

7. CONCLUZII

7.1. CONCLUZII

Sistemul medical veterinar este în continuă expansiune și proces de digitalizare, cu scopul de a ușura atât munca veterinarilor, cât și a posesorilor de animale de companie. Cu ajutorul aplicației, utilizatorii nu vor mai avea nevoie să consulte periodic carnetele de sănătate ale animalelor, deoarece vor avea toate informațiile la un clic distanță. La acestea se adaugă notificările prin email, care nu se pot pierde la fel de ușor precum cele de tip „push”, de pe telefoanele mobile.

În cadrul prezentei lucrări, pornind de la o nevoie avută, nu doar de mine, ci și de alte persoane chestionate, am transformat-o într-o idee și o dorință personală de a avea un impact pozitiv asupra comunității iubitorilor de animale de companie. Consider că, odată cu dezvoltarea aplicației „PawsPlan”, am rezolvat această nevoie semnalizată de o comunitate foarte mare.

Aplicația are o interfață simplă, plăcută privitorilor, fiind ușor și intuitiv de utilizat. Acest aspect este relevant pentru orice categorie de utilizatori, dar, mai ales pentru persoanele în vârstă. În general, acestea au cele mai mari dificultăți în a reține anumite acțiuni, dar și în a utiliza aplicații mult prea complexe, deci „PawsPlan” este potrivită chiar și pentru această categorie de vârstă.

Întreg procesul de creare a aplicației a fost unul complex și a suferit diverse modificări, din ambiția de a învăța tehnologii noi sau din conștientizarea că nu toate funcționalitățile gândite inițial aveau obiective realiste în perioada de timp alocată.

În tot acest timp, am învățat să folosesc serviciile Firebase, am realizat că o aplicație web poate fi perfect funcțională chiar și fără un server și mi-am îmbunătățit cunoștințele pe care le aveam legate de framework-ul ReactJS.

Consider că acest proiect mi-a oferit contextul de a mă familiariza cu procesul de dezvoltare a unei aplicații, de la stadiul de idee și până la stadiul final, mi-a arătat că orice programator se lovește de anumite obstacole în parcursul său, dar, cu pregătirea necesară și cu ambiția de a depăși impedimentele, poate avea un impact pozitiv în societate prin aplicațiile sale.

7.2. DIRECȚII DE DEZVOLTARE

Chiar dacă versiunea actuală a aplicației „PawsPlan” este pregătită de utilizare, cu siguranță aceasta are loc de îmbunătățiri.

Un prim pas ar fi disponibilitatea aplicației pe dispozitivele mobile, pentru a-i crește nivelul de accesibilitate. Totuși, în continuare consider că notificările trimise ar trebui să fie prin email, deoarece sunt mai ușor de urmărit și nu se pot șterge la fel de ușor ca cele de tip „push”.

Acest lucru s-ar putea face printr-un serviciu automat de trimis mailuri, care să ruleze zilnic. O posibilă implementare, mulată pe arhitectura actuală a aplicației, este

folosind o combinație a serviciului Firebase Schedule Functions și a extensiei Firebase Trigger Email.

O funcționalitate ce ar aduce și veterinarilor în mediul aplicației ar fi posibilitatea ca aceștia să își adauge disponibilitatea, iar utilizatorii să poată face programări la consult chiar din aplicația „PawsPlan”.

Pentru a îmbunătăți relaționarea dintre posesorii de animale de companie și medicii veterinari în cadrul aplicației, o direcție potrivită de dezvoltare este cea în care veterinarilor pot adăuga medicațiile pentru un animal direct în „PawsPlan” sau pot chiar trimite rezultatele investigațiilor (analize, ecografii, radiografii) către conturile stăpânilor.

Legat de lista de veterinari parteneri, aceasta poate fi în continuă expansiune, iar de aceea s-ar putea ca aplicația să afișeze doar cabinetele din apropiere, bazat pe locația utilizatorului.

BIBLIOGRAFIE

- [1] Haverbeke, M., *Eloquent JavaScript*, Third edition, 2018
- [2] Monorey, L., *The Definitive Guide to Firebase*, Seattle, Washington, USA, 2017
- [3] Raggett, D., Le Hors, A., Jacobs, I., *HTML 4.0 Specification*, W3C Recommendation, revizuită în 24 Aprilie 1998
- [4] Rawat, P., Mahajan, A. N., *ReactJS: A Modern Web Development Framework*, International Journal of Innovative Science & Research Technology, Volume 5, Issue 11, November 2020
- [5] Trammell, J. P., *The Effect of Therapy Dogs on Exam Stress and Memory*, Anthrozoös, 2017
- [6] Yadav, P., Barwal, P.N., *Designing Responsive Websited Using HTML And CSS*, International Journal of Scientific & Technology Research, Volume 3, Issue 11, November 2014
- [7]***, *10 Famous HTML-only Website Examples*,
adresa: <https://devpractical.com/html-only-websites/> (accesare lunie 2023)
- [8]***, *The Beginner's Guide to Responsive Web Design*,
adresa: <https://kinsta.com/blog/responsive-web-design/> (accesare lunie 2023)
- [9]***, *2022 Developer Survey*,
adresa: <https://survey.stackoverflow.co/2022/> (accesare lunie 2023)
- [10]***, *The History of React.js on a Timeline*,
adresa: <https://blog.risingstack.com/the-history-of-react-js-on-a-timeline/> (accesare lunie 2023)
- [11]***, *Top JavaScript Frameworks and Technology 2023*,
adresa: <https://medium.com/javascript-scene/top-javascript-frameworks-and-technology-2023-4e4a06d6be93> (accesare lunie 2023)
- [12]***, *What is React JS?*,
adresa: <https://blog.effectussoftware.com/what-is-react-js/> (accesare lunie 2023)
- [13]***, *Push Notifications vs Email: Which Is Better? (Pros and Cons)*,
adresa: <https://www.wpbeginner.com/beginners-guide/push-notifications-vs-email-which-is-better/> (accesare lunie 2023)
- [14]***, *Push Notifications*,
adresa: <https://vwo.com/push-notifications/> (accesare lunie 2023)
- [15]***, *How to send reminder email for upcoming meeting?*,
adresa: <https://www.boostsolutions.com/articles/how-to-send-reminder-email-for-upcoming-meeting.html>
- [17]***, *Firebase*,
adresa: <https://firebase.google.com/> (accesare lunie 2023)
- [18]***, *Developer documentation for Firebase*,
adresa: <https://firebase.google.com/docs/> (accesare lunie 2023)
- [19]***, *Firebase Products / Build*,
adresa: <https://firebase.google.com/products-build> (accesare lunie 2023)

[20]***, *What is Git and Why Should You Use It?*,

adresa: <https://www.nobledesktop.com/learn/git/what-is-git> (accesare Iunie 2023)

[21]***, *Use Font Awesome on the Web*,

adresa: <https://fontawesome.com/v5/docs/web/use-with/react> (accesare Iunie 2023)

LISTA FIGURILOR ȘI A TABELELOR

Figura 1 – Profilul nimalului de companie în aplicația „VitusVet”

Figura 2 – Medicația pentru un animal de companie în aplicația „Furiend – Pet Health Tracker”

Figura 3 – Elementele unei notificări „push”

Figura 4 – Exemplu de notificare prin email, de tip reminder

Figura 5 – Prima pagină web, „The WWW Project”, creată doar cu limbajul HTML

Figura 6 – Folosirea media queries, parte a CSS3, pentru adaptarea la ecrane

Figura 7 – Cele mai iubite limbaje de programare, conform „Stack Overflow Developer Survey”, din 2022

Figura 8 – Menționarea frameworkurilor de front-end în descrierile job-urilor

Figura 9 – Căutările de frameworkuri de front-end în 2022

Figura 10 – Descărcările de frameworkuri de front-end în 2022

Figura 11 – Satisfacția utilizatorilor pentru frameworkuri de front-end în 2022

Figura 12 – Modalități de autentificare disponibile în Firebase Authentication

Figura 13 – Diagrama UML a cazurilor de utilizare a aplicației PawsPlan

Figura 43 – Diagrama de flux al utilizatorului în aplicația PawsPlan

Figura 15 – Structura unui document aflat în colecția de utilizatori

Figura 16 – Structura unui document aflat în colecția de animale

Figura 17 – Structura unui document aflat în colecția de medicații

Figura 18 – Structura unui document aflat în colecția de veterinari

Figura 19 – Înregistrarea utilizatorului în aplicație

Figura 20 – Secvența de cod JSX ce conține formularul de autentificare

Figura 21 – Secvența de extragere a datelor din Firestore și crearea vectorului de animale

Figura 22 – Accesarea proprietăților obiectului prin `{selectedPet.petName}`, `{selectedPet.age}`, `{selectedPet.breed}`

Figura 23 – Secvența de cod ce conține Sweet Alert-ul de confirmare a operației

Figura 24 – Adăugarea animalului în baza de date și redirectionarea către pagina principală

Figura 25 – Funcția de redirectare, ce construiește adresa URL dorită

Figura 26 – Interogarea în baza de date pentru a crea lista de medicații

Figura 27 – Afișarea listei de medicații pentru un animal selectat

Figura 28 – Secvența de cod de adăugare a unei medicații specifice unui animal

Figura 29 – Afișarea listei cu date despre veterinari, prin maparea vectorului

Figura 30 – Funcția ce declanșează trimiterea emailurilor către utilizatori

Figura 44 – Inserarea variabilelor `to_name` și `to_email` în șablonul de email din EmailJS

Figura 32 – Prima pagină, în cazul în care utilizatorul nu este autentificat

Figura 33 – Formularul de înregistrare

Figura 34 – Formularul de conectare

Figura 35 – Lista cu numele animalelor și detaliile animalului selectat

Figura 36 – Formularul de adăugare a unui animal

Figura 37 – Formularul de adăugare a unui animal

Figura 38 – Alertarea utilizatorului cu privire la ștergerea unui animal

Figura 39 – Lista de medicații ale unui animal selectat

Figura 40 – Formularul de adăugare a unei medicații

Figura 41 – Pagina cu detalii despre cabinetele veterinare partenere

Figura 42 – Conținutul emailului trimis automat către utilizatori

Tabel 2 - Analiza aplicațiilor „VitusVet” și „Furiend – Pet Health Tracker” comparativ cu „PawsPlan”; funcționalitățile prezente sunt marcate cu ✓, iar cele absente cu ✕

**DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR***

Subsemnatul BELEA F.C. BIANCA-MARIA

legitimată cu CI seria TZ nr. 772897

CNP 600 12 11 35 00 16

autorul lucrării PAWISPLAN - APLICATIE DE MONITORIZARE A
TRATAMENTELOR ANIMALELOR DE COMPANIE

elaborată în vederea susținerii examenului de finalizare a studiilor de
LICENȚĂ organizat de către Facultatea

DE AUTOMATICĂ ȘI CALCULATOARE din cadrul Universității

Politehnica Timișoara, sesiunea Iunie 2023 a anului universitar

2022-2023, coordonator Ș. DR. ING. COSMIN MARSAVINA, luând în

considerare art. 34 din Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării ~~am utilizat~~ instrumente specifice inteligenței artificiale (IA) și anume _____ (denumirea) _____ (sursa), pe care le-am citat în conținutul lucrării/nu am utilizat instrumente specifice inteligenței artificiale (IA)¹.

Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

24.06.2023

Semnătura

76

*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

¹ Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA